Computers and Operations Research 113 (2020) 104782

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

Strong bounds for resource constrained project scheduling: Preprocessing and cutting planes

Janniele A.S. Araujo[a,b,c,*], Haroldo G. Santos[a,d], Bernard Gendron[e,c], Sanjay Dominik Jena[f,c], Samuel S. Brito[a,b], Danilo S. Souza[a]

[a] Department of Computing, DECOM, Universidade Federal de Ouro Preto, Brazil
[b] Department of Computing and Systems, DECSI, Universidade Federal de Ouro Preto, Brazil
[c] Interuniversity Research Centre on Enterprise Networks, Logistics and Transportation, CIRRELT, Canada
[d] Computer Science Department, CODeS, KU Leuven, Belgium
[e] Department of Computer Science and Operations Research, Université de Montréal, Canada
[f] Department of Management and Technology, École des Sciences de la Gestion, Université de Québec à Montréal, Canada

ARTICLE INFO

Article history:
Received 24 January 2019
Revised 4 August 2019
Accepted 28 August 2019
Available online 29 August 2019

Keywords:
Resource constrained project scheduling
Cutting planes
Mixed-integer linear programming
Preprocessing

ABSTRACT

Resource Constrained Project Scheduling Problems (RCPSPs) without preemption are well-known $\mathcal{NP}$-hard combinatorial optimization problems. A feasible RCPSP solution consists of a time-ordered schedule of jobs with corresponding execution modes, respecting precedence and resources constraints. In this paper, we propose a cutting plane algorithm to separate five different cut families, as well as a new preprocessing routine to strengthen resource-related constraints. New lifted versions of the well-known precedence and cover inequalities are employed. At each iteration, a dense conflict graph is built considering feasibility and optimality conditions to separate cliques, odd-holes and strengthened Chvátal-Gomory cuts. The proposed strategies considerably improve the linear relaxation bounds, allowing a state-of-the-art mixed-integer linear programming solver to find provably optimal solutions for 754 previously open instances of different variants of the RCPSPs, which was not possible using the original linear programming formulations.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

This paper proposes automatic Mixed-Integer Linear Programming (MILP) reformulation strategies for non-preemptive Resource Constrained Project Scheduling Problems (RCPSPs) and its variants. From a theoretical point of view, RCPSPs are challenging combinatorial optimization problems, classified as $\mathcal{NP}$-hard (Blazewicz et al., 1983; Garey and Johnson, 1979). These problems cover a wide range of applications, such as particle therapy for cancer treatment in healthcare (Riedler et al., 2020), civil engineering (Liu and Wang, 2008), manufacturing and assembly of large products (Liu et al., 2014), as well as development and launching of complex systems (Demeulemeester and Herroelen, 2002). Comprehensive reviews on RCPSPs can be found, for example, in Artigues et al. (2008), Demeulemeester and Herroelen (2002), Schwindt and Zimmermann (2015).

In this paper, we consider the following problem variants of the RCPSP, from the most specific one to the most generalized version:

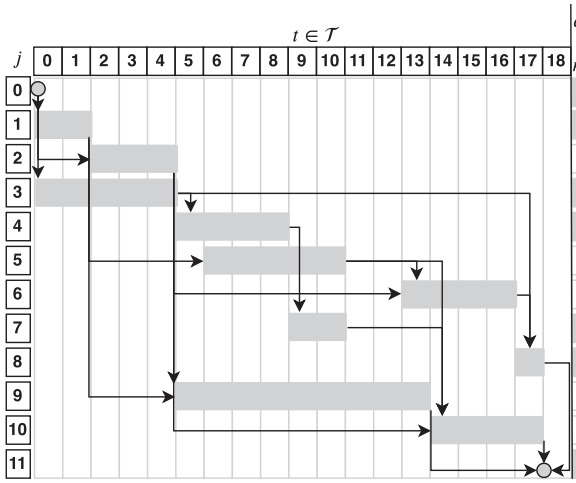
SMRCPSP: single-mode resource-constrained project scheduling problem;
MMRCPSP: multi-mode resource-constrained project scheduling problem;
MMRCMPSP: multi-mode resource-constrained multi-project scheduling problem.

The SMRCPSP is the simplest variant and involves only one processing mode for each job. A feasible solution consists in the assignment of jobs at specific time periods over a planning horizon, respecting precedence and resource usage constraints. The resources in the SMRCPSP are renewable at each time period.

In the MMRCPSP, it is possible to choose between different job processing modes, each of them having different durations and consuming different amounts of resources. Two types of resources are available: the renewable resources at each time period and the non-renewable ones available for the entire project execution. While the use of renewable resources only impacts the

* Corresponding author at: Department of Computing and Systems, DECSI, Universidade Federal de Ouro Preto, Brazil.
E-mail address: janniele@ufop.edu.br (J.A.S. Araujo).

https://doi.org/10.1016/j.cor.2019.104782
0305-0548/© 2019 Elsevier Ltd. All rights reserved.



	duration d_j			resources: renewable r_0 9			r_1 8			non − renewable k_0 35			k_1 31		
j	m_0	m_1	m_2	m_0	m_1	m_2	m_0	m_1	m_2	m_0	m_1	m_2	m_0	m_1	m_2
0	0	-	-	0	-	-	0	-	-	0	-	-	0	-	-
1	2	6	8	0	0	0	4	4	3	3	0	0	0	7	7
2	2	3	7	0	0	0	10	5	4	4	3	0	0	0	7
3	5	9	10	3	1	0	0	0	8	0	7	0	7	0	6
4	4	4	6	7	6	4	0	0	0	8	0	8	0	8	0
5	2	3	5	0	0	2	4	4	0	6	0	1	0	3	0
6	1	4	9	10	0	6	0	5	0	10	0	0	0	4	4
7	2	6	10	5	0	4	0	7	0	0	7	6	8	0	0
8	1	4	6	6	6	3	0	0	0	0	10	10	6	0	0
9	6	8	8	10	0	9	0	4	0	0	6	0	4	0	3
10	4	5	5	4	0	3	0	9	0	0	0	8	7	4	0
11	0	-	-	0	-	-	0	-	-	0	-	-	0	-	-

Fig. 1. An optimal solution for instance *j*102_4.*mm* and its characteristics.

delay/speedup of the projects, the use of non-renewable resources can produce infeasible solutions.

A generalization of the previous problem variants to handle multiple projects and global renewable resources is the MMRCMPSP. While the previous two problem variants, in their objective (minimization) functions, consider the makespan, i.e., the total length of the schedule to finish all jobs, an important modeling feature of this generalization is an additional objective: the project delays from the Critical Path Duration (CPD). The CPD is a theoretical lower bound on the earliest finishing time ($\breve{e}_p^f$) of project p, computed by the Critical Path Method (CPM) (Kelley and Walker, 1959) and disregarding any resource constraints.

Fig. 1 illustrates the example instance *j*102_4.*mm*[1], a small instance for the MMRCPSP variant. The figure shows the characteristics of this instance and one optimal solution. We will refer to this example instance to explain the cutting planes presented in the next sections.

The instance illustrated in Fig. 1 has twelve jobs. The first and the last are artificial jobs representing the start and the end of the project. Each job j can be processed in three different modes m. Artificial jobs 0 and 11 have only one execution mode that does not consume resources and have duration 0; thus columns m_1 and m_2 are filled with "-". Two renewable resources $\{r_0, r_1\}$ with capacities {9, 8} and two non-renewable resources $\{k_0, k_1\}$ with capacities {35, 31} are available. This figure provides information about resource consumption and duration of job j processing on mode m. The Gantt chart in the figure shows the starting time allocation and duration of jobs for the optimal solution found for this single project. Arcs represent the precedence relationship between jobs. Values emphasized in grayscale represent the active modes for this solution. The CPD corresponds to value 15. Note that the earliest starting time ($\breve{e}_j^s$) of job 5 corresponds to 2, but due to the availability of resources at this time period, its allocation had to be postponed. The makespan is 18 time units.

In this paper, we propose a new preprocessing technique to improve resource-related constraints by strengthening their coefficients using known information about different renewable, non-renewable resources and precedence constraints. We also propose a cutting plane algorithm employing five different cut separation algorithms. Conflict-based cuts such as cliques and odd-holes are generated considering an implicit dense conflict graph, which is updated at each iteration considering optimality and feasibility conditions. This conflict graph is also used in a MILP separation of the Chvátal-Gomory cuts to produce stronger inequalities. Furthermore, new lifted versions of the well-known disaggregated precedence cuts and cover cuts are separated. The contribution on the linear relaxation bound provided by each cut family is examined on experiments on a large set of benchmark instances of the three problem variants considered in this paper. Overall, stronger bounds were obtained, allowing a state-of-the-art MILP solver to find provably optimal solutions for 755 previously open instances, which was not possible using the original linear programming formulations.

The paper is organized as follows. Section 2, reviews the literature related to our work. Section 3 introduces the integer programming formulation used along with some simple input data preprocessing routines based on upper bounds. The new strengthening procedure for renewable resources constraints is also presented in this section. Section 4 outlines the proposed cutting plane method, followed by a detailed description of each cut generation routine. In Section 5, the computational results are presented. Finally, in Section 6, we conclude our work and discuss future research directions.

2. Literature review

Various MILP based formulations have been proposed in the literature to model resource constrained project scheduling problems. Pritsker et al. (1969) proposed the first binary programming formulation where variables x_{jt} indicate whether a job j ends at time t ($x_{jt} = 1$) or not ($x_{jt} = 0$). This formulation is known as the discrete-time or time-indexed formulation. The number of binary decision variables in this formulation is related to an upper bound $\bar{t}$ for the number of time periods required to complete the project. Thus the number of variables is $\mathcal{O}(n \times \bar{t})$, where n is the number of jobs. Kolisch and Sprecher (1996) extended this formulation to handle different execution modes, adding one additional index m to the binary variables and incorporating this index in the resource-related constraints. In Kone et al. (2011), a new formulation was proposed based on events called on/off event-based (OOE) with $\mathcal{O}(n^2)$ variables.

Time-indexed formulations have been extensively studied and applied to the RCPSP (see, for example, Baptiste and Demassey, 2004; Christofides et al., 1987; de Souza and Wolsey, 1997; Demassey et al., 2005; Hardin et al., 2008; Sankaran et al., 1999). Six time-indexed formulations were studied in Artigues (2017). These

[1] http://www.om-db.wi.tum.de/psplib/files/j10.mm.tgz.

formulations are initially categorized into three groups according to the meaning of variables, as follows:

Pulse: pulse discrete time - PDT (the most used) formulation with binary variables x_{jt} $\forall j \in \mathcal{J}, \forall t \in \mathcal{T}$, such that $x_{jt} = 1$ if job j starts at time t, otherwise $x_{jt} = 0$;

Step: step discrete time - SDT formulation with binary variables y_{jt} $\forall j \in \mathcal{J}, \forall t \in \mathcal{T}$, such that $y_{jt} = 1$ if job j starts at time t or before, otherwise $y_{jt} = 0$;

On/Off: on/off discrete time - OODT (the lesser used) formulation with binary variables z_{jt} $\forall j \in \mathcal{J}, \forall t \in \mathcal{T}$ such that $z_{jt} = 1$ if job j is processed at time t, otherwise $z_{jt} = 0$.

For each one of these categories, it is possible to model the precedence constraints in a weak or strong way according to the linear programming (LP) relaxation strength.

Aggregated: this is a weak way to model precedence constraints, using variable coefficients greater than or equal to 1; aggregated constraints generate $\mathcal{O}(n^2)$ inequalities, each one with $\mathcal{O}(\bar{t} \times m)$ variables, where m is the maximum number of modes;

Disaggregated: this is a stronger way to model precedence constraints, using variable coefficients equal to $\{-1, 1\}$; disaggregated constraints generate $\mathcal{O}(n^2 \times \bar{t})$ inequalities, each one with $\mathcal{O}(\bar{t} \times m)$ variables.

Artigues (2017) concluded that PDDT (pulse disaggregated discrete time), SDDT (step disaggregated discrete time) and OODDT (on/off disaggregated discrete time), which are formulations with disaggregated constraints, are all equivalent in terms of the strength of their LP-relaxations and belong to the family of strong time-indexed formulations. He also concluded that the formulations with their aggregated counterparts, PDT, SDT, and OODT, belong to a family of weak formulations and are also all equivalent in terms of their LP relaxation.

Even though the aggregated constraints are weaker, they are much less dense. For this reason, papers from the literature (see, for example, Christofides et al., 1987; Zhu et al., 2006), begin the formulation with the aggregated constraints and add the disaggregated ones as cutting planes. As in previous works, our algorithm starts with the weak time-indexed formulation pulse discrete time (PDT) based on formulations proposed in Kolisch and Sprecher (1996), Pritsker et al. (1969) for the MMRCMPSP version. In the next paragraphs, we review some computational approaches to handle MILP formulations for the RCPSPs.

Most of the exact algorithms for the RCPSPs (Brucker et al., 1998; Chakrabortty et al., 2015; Christofides et al., 1987; Demeulemeester and Herroelen, 1992) are built upon LP based Branch and Bound (B&B) (Christofides et al., 1987; Land and Doig, 2010) algorithms. A key component in the design of these algorithms is which formulation is employed: compact formulations, with a polynomial number of variables and constraints, are usually able to quickly provide valid lower bounds since their LP relaxations are easily solved. The LP relaxation bounds are relatively weak but can be significantly improved by adding cutting planes in a Branch-and-Cut (B&C) algorithm.

Cutting planes and strengthening constraints in MILP models are explored for RCPSP and to other variants of scheduling problems. Applegate and Cook (1991) proposed cliques cuts, half cuts, and other cuts specific to job-shop scheduling problems (JSSP). Hardin et al. (2008) proposed a lifting procedure to cover-clique inequalities to a resource-constrained scheduling problem with uniform resources (URCSP) requirement. Cavalcante et al. (2001) applied cover cuts to the labor constrained scheduling problem (LCSP), based on practical requirements arising in industry.

Sankaran et al. (1999) proposed a cutting-plane algorithm for the SMRCPSP with minimal cover inequalities and clique inequalities to test problems provided by Patterson and Huber (1974). Also, they introduced three preprocessing techniques: reduction of lower and upper bounds; the identification of redundant constraints between resource and precedence constraints, and the coefficient strengthening in constraints (Johnson et al., 1985).

Christofides et al. (1987) proposed a B&B algorithm that uses disjunction arcs to handle resource conflicts. Four lower bounds are examined: the first one is based on the longest path in the precedence graph; the second one is based on an LP relaxation strengthened with cuts; the third one is based on Lagrangian relaxation and the fourth one is based on disjunctive arcs. Their LP-based method incorporated the dynamic inclusion of disaggregated precedence constraints and resource-based conflict constraints for pairs of jobs. The first lower bound, based on precedence constraints, can be computed quickly and was used in the B&B algorithm with other bounds. The second lower bound was promising, but was not used within the B&B method. The Lagrangian relaxation technique was found to provide less competitive results. They provide additional inequalities for the time-indexed version. Finally, the fourth lower bound performed quite well, especially for problems with tight resource constraints. For the SMRCPSP, Christofides et al. (1987) were able to prove the optimality of instances involving up to 25 jobs and 3 resources.

Zhu et al. (2006) presented a B&C algorithm for the MMRCPSP, including cuts derived from resource conflicts, where all resource constraints are in the form of generalized upper bound (GUB) constraints. Besides, disaggregated cuts from the precedence relationship for pairs of jobs (j, s) in the precedence graph are included. To speedup the solution process, an adaptive branching scheme is developed along with a bound adjustment scheme that is always executed iteratively after branching. To optimize the solutions found in the first stage, the authors use a high-level neighborhood search strategy called Local Branching (Fischetti and Lodi, 2007). For the MMRCPSP, the authors were able to prove the optimality 554 of instances with 20-jobs and 506 instances with 30-jobs.

Different techniques such as Constraint Programming (CP) and Satisfiability Solving (SAT) have also been used to solve the MMRCPSP. In this context, Demassey et al. (2005) uses CP techniques to provide valid inequalities to strengthen LP relaxations. A recent work using cutting planes as valid clauses for SAT is presented in Schnell and Hartl (2017). They propose three formulations based on Constraint Programming to solve the MMRCPSP, using the G12 CP platform and the Solving Constraint Integer Programs (SCIP) as an optimization framework, both making use of solution techniques combining CP and SAT. They further combine MILP by inserting a new global constraint on the domain of renewable resources for SCIP. The authors achieved the same results with better computational times than (Zhu et al., 2006). They also were able to prove the optimality of 1428 instances with 50-jobs and 100-jobs and to improve various lower and upper bounds.

3. Integer programming formulation

This section introduces the time-indexed formulation based upon the discrete time formulations proposed in Kolisch and Sprecher (1996), Pritsker et al. (1969). The most common objective function for the RCPSP is the makespan (Artigues et al., 2008; Demeulemeester and Herroelen, 2002; Kolisch and Sprecher, 1996; Pritsker et al., 1969; Zhu et al., 2006) minimizing the total schedule duration required to finish all jobs. Wauters et al. (2016) proposed a hierarchical objective to minimize the Total Project Delay (TPD) and the total makespan (TMS). The former, denoted formally

in Eq. (1), is the main objective. The latter, given in Eq. (2), is a tiebreaker.

$$TPD = \sum_{p \in P} PD_p \tag{1}$$

$$TMS = \max_{p \in \mathcal{P}} f_p - \min_{p \in \mathcal{P}} \sigma_p \tag{2}$$

The project delay (PD_p) for a project $p \in \mathcal{P}$ is defined as the difference between its CPD_p (which does not consider any resource constraints), and its makespan $MS_p = f_p - \sigma_p$, the actual project duration taking into consideration resource constraints, the finishing time f_p and the release date σ_p of this project.

$$PD_p = MS_p - CPD_p \tag{3}$$

To work with all three problem variants in a unified way, we always report our results considering the TPD, which is generic enough to handle all problem variants considered in this paper.

3.1. Input data

The following notation is used throughout this paper to describe the input data:

- $\mathcal{P}$: set of all projects;
- $\mathcal{J}$: set of all jobs;
- $\mathcal{M}_j$: set of modes available for job $j \in \mathcal{J}$;
- $\mathcal{J}_p$: set of jobs belonging to project p, such that $\mathcal{J}_p \subseteq \mathcal{J}\ \forall p \in \mathcal{P}$;
- $\mathcal{K}$: set of non-renewable resources;
- $\mathcal{R}$: set of renewable resources;
- $\mathcal{S}$: set of direct precedence relationships between two jobs $(j, s) \in \mathcal{J} \times \mathcal{J}$;
- $\mathcal{T} \subset \mathbb{Z}^+$: set of time periods in the planning horizon for all projects $p \in \mathcal{P}$;
- $\mathcal{T}_{jm} \subset \mathcal{T}$: time horizon for each job $j \in \mathcal{J}$ on mode $m \in \mathcal{M}_j$, defined after preprocessing;
- $d_{jm} \in \mathbb{Z}^+$: duration of job $j \in \mathcal{J}$ on mode $m \in \mathcal{M}_j$;
- $q_{kjm} \in \mathbb{Z}^+$: required amount of non-renewable resource $k \in \mathcal{K}$ to execute job $j \in \mathcal{J}$ on mode $m \in \mathcal{M}_j$;
- $q_{rjm} \in \mathbb{Z}^+$: required amount of renewable resource $r \in \mathcal{R}$ to execute job $j \in \mathcal{J}$ on mode $m \in \mathcal{M}_j$;
- $\check{q}_k \in \mathbb{Z}^+$: available amount of non-renewable resource $k \in \mathcal{K}$;
- $\check{q}_r \in \mathbb{Z}^+$: available amount of renewable resource $r \in \mathcal{R}$;
- $\sigma_p \in \mathcal{T}$: release date of project p;
- $a_p \in \mathcal{J}_p$: artificial job belonging to project $p \in \mathcal{P}$, which represents the end of the project.

3.2. Preprocessing input data

An effective way to reduce the search space is by identifying tight time windows in which it is valid to process jobs. A basic technique to define the earliest starting time $\check{e}^s_j$ for jobs $j \in \mathcal{J}$ consists of computing the CPD using CPM Kelley and Walker (1959) without considering resource constraints. This methods allows to compute the $\check{e}^s_j$ of all jobs, taking into consideration the precedence relationships. The longest path of a project, also known as the critical path, provides a lower bound for the completion time of each project.

Consider, for each project $p \in \mathcal{P}$, the release date σ_p, and lower bound based (i.e., the length of the critical path) λ_p as input data and the value β_p, an upper bound for each project p, obtained from any feasible solution. Optimality conditions can be used to restrict the set of valid time periods when a job can be allocated. We initially consider the value α computed by Eq. (4), that represents an upper bound to the maximum total project delay allowed.

$$\alpha = \sum_{p \in \mathcal{P}} (\beta_p - \sigma_p - \lambda_p) \tag{4}$$

Thus, the maximum time period $\check{t} \in \mathcal{T}$ that needs to be considered int the planning, can be obtained by Eq. (5).

$$\begin{aligned} &\check{t} = \max_{p \in P} (\sigma_p + \lambda_p + \alpha) \\ &\mathcal{T} = \{0, \ldots, \check{t}\} \end{aligned} \tag{5}$$

Analogously, upper bounds can be computed for processing times of jobs. The upper bounds can be strengthened if the selection of modes with different durations is also considered. The upper bounds are used, along with the duration of each job and without considering the resource constraints, to define the latest starting times ($\check{l}^s_j$) for jobs $j \in \mathcal{J}$. A job j from a project p when processed at mode m will push forward (i.e., postpone) all successor jobs by exactly d_{jm} time units. Consider set $\overline{\mathcal{S}}_j$, containing the entire chain of successors of job j on the longest path from job j to the artificial job a_p (indicating the project completion). Let lower bound $\mathcal{L}_{jm}$ be the total duration in this path, computed considering only the fastest processing modes for each job in this chain. The maximum allocation time or latest starting time ($\check{l}^s_{jm}$) for a job j from a project p when processing on mode m to $\mathcal{T}_{jm}$ is given by Eq. (6).

$$\begin{aligned} &\check{l}^s_{jm} = \sigma_p + \lambda_p - \mathcal{L}_{jm} + \alpha \\ &\mathcal{T}_{jm} = \{\check{e}^s_j, \ldots, \check{l}^s_{jm}\} \end{aligned} \tag{6}$$

Similar bounds can be derived for any two jobs in this path also considering the fastest processing modes for all jobs except the first one:

- $\check{d}_{jms}$: the shortest path in the precedence graph considering the length of the arcs between job j and successor job $s \in \overline{\mathcal{S}}_j$ considering mode $m \in \mathcal{M}_j$;
- $\check{d}^*_{js}$: the shortest path in the precedence graph considering the length of the arcs between job j and successor job $s \in \overline{\mathcal{S}}_j$ considering j fastest mode.

3.3. Formulation

Binary decision variables are used to select the mode and starting times for the jobs. They are defined as follows:

$$x_{jmt} = \begin{cases} 1 & \text{if job } j \in \mathcal{J} \text{ is allocated on mode } m \in \mathcal{M}_j \\ & \text{at starting time } t \in \mathcal{T}_{jm}; \\ 0 & \text{otherwise.} \end{cases}$$

We introduce in this formulation on/off discrete time variables studied in Artigues (2017) to allow resources constraints and cutting planes, detailed in the next sections, to be expressed with fewer variables. The following binary decision variables indicate during which time periods jobs are being processed:

$$z_{jmt} = \begin{cases} 1 & \text{if the job } j \in \mathcal{J} \text{ is allocated on mode } m \in \mathcal{M}_j \text{ and} \\ & \text{is being processed during time } t \in \mathcal{T}_{jm}; \\ 0 & \text{otherwise.} \end{cases}$$

The objective function minimizes the total project delay over the project completion times for projects and their critical paths. Consider the following integer variable included in the objective function:

- $h \in \mathbb{Z}^+$: integer variable used to compute the makespan, included in the objective function with a small coefficient ϵ to break ties.

Minimize:

$$\sum_{p \in \mathcal{P}} \sum_{m \in \mathcal{M}_{a_p}} \sum_{t \in \mathcal{T}_{a_p m}} [t - (\sigma_p + \lambda_p)] x_{a_p mt} + \epsilon h \tag{7}$$

subject to:

$$\sum_{m \in \mathcal{M}_j} \sum_{t \in \mathcal{T}_{jm}} x_{jmt} = 1 \qquad \forall j \in \mathcal{J} \tag{8}$$

$$\sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}_j}\sum_{t\in\mathcal{T}_{jm}} q_{kjm}x_{jmt} \le \check{q}_k \qquad \forall k\in\mathcal{K} \quad (9)$$

$$\sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}_j} q_{rjm}z_{jmt} \le \check{q}_r \qquad \forall r\in\mathcal{R}, \forall t\in\mathcal{T} \quad (10)$$

$$\sum_{m\in\mathcal{M}_j}\sum_{t\in\mathcal{T}_{jm}} (t+d_{jm})x_{jmt} - \sum_{z\in\mathcal{M}_s}\sum_{i\in\mathcal{T}_{sz}} ix_{szi} \le 0 \quad \forall j\in\mathcal{J}, \forall s\in\mathcal{S}_j \quad (11)$$

$$z_{jmt} - \sum_{t'=(t-d_{jm}+1)}^{t} x_{jmt'} = 0 \qquad \forall j\in\mathcal{J}, \forall m\in\mathcal{M}_j, \forall t\in\mathcal{T}_{jm} \quad (12)$$

$$h - \sum_{m\in\mathcal{M}_{a_p}}\sum_{t\in\mathcal{T}_{a_pm}} tx_{a_pmt} \ge 0 \qquad \forall p\in\mathcal{P} \quad (13)$$

$$x_{jmt} \in \{0,1\} \qquad \forall j\in\mathcal{J}, \forall m\in\mathcal{M}_j, \forall t \quad (14)$$

$$z_{jmt} \in \{0,1\} \qquad \forall j\in\mathcal{J}, \ \forall m\in\mathcal{M}_j, \forall t\in\mathcal{T}_{jm} \quad (15)$$

$$h \ge 0 \quad (16)$$

Constraints (8) ensure that each job is allocated to exactly one starting time and one mode. Constraints (9) and (10) are capacity constraints for non-renewable and renewable resources, respectively. Constraints (11) force precedence relationships to be satisfied. Constraints (12) link variables z and variables x. Constraints (13) compute the total makespan. Finally, constraints (14), (15) and (16) respectively ensure that variables x and z can only assume binary values and h can only assume nonnegative values.

3.4. Preprocessing MILP formulation

Johnson et al. (1985) introduce an interesting preprocessing method to strengthen constraint coefficients using the knapsack structure of resource constraints. This preprocessing was used in Sankaran et al. (1999) for the SMRCPSP and analyzes one resource usage constraint at time. In this paper, we propose a preprocessing technique that considers various constraints (precedence and the usage of other renewable and non-renewable resources), besides the renewable resource constraint, which will be strengthened.

The proposed procedure to strengthen resource usage constraints (10) was inspired by Fenchel cutting planes Boyd (1992, 1994). Fenchel cutting planes are based on the enumeration of incidence vectors to find the most violated inequality for a subset of binary variables. In our paper, we enumerate feasible subsets of jobs and modes to create a linear problem to find the best possible strengthening of a given resource constraint.

The strengthening procedure is presented in Algorithm 1. First, it computes, for each t, a set $\mathcal{G}_t$ composed of all jobs and modes (j, m) available for processing at time $t\in\mathcal{T}_{jm}$ (see Algorithm 1, lines 1–2). Formally, these sets can be computed as stated in Eq. (17).

$$\mathcal{G}_t = \{\ (j,m) \ \in \ \mathcal{J}\times\mathcal{M}_j \mid \ t\in\mathcal{T}_{jm}\} \quad (17)$$

To illustrate the coefficient strengthening technique, consider for $t=4$, the jobs and modes that make up the set $\mathcal{G}_4 = \{(1,0),(1,1),(1,2),(2,1),(2,2),(3,0),(5,0),(5,1),(5,2),(6,1),(6,2),(9,1),(9,2)\}$. Consider also, the following original constraint restricting the usage of renewable resource r_0 at time $t=4$:

$$0z_{1,0,4}+0z_{1,1,4}+0z_{1,2,4}+0z_{2,1,4}+0z_{2,2,4}+3z_{3,0,4}+0z_{5,0,4}$$
$$+0z_{5,1,4}+2z_{5,2,4}+0z_{6,1,4}+6z_{6,2,4}+0z_{9,1,4}+9z_{9,2,4} \le 9.$$

The next step is to enumerate all valid combinations of jobs and modes (j, m) that can be processed in parallel at time t, i.e., that satisfy all resources constraints and do not have precedence relations among each other. Mingozzi et al. (1998) designated these valid combinations as *feasible subsets*.

Algorithm 1: Strengthening_procedure.

Data: RCPSP model $\mathcal{M}$, *it* iteration limit, *tl* time limit, set $\mathcal{J}$, set $\mathcal{M}_j$, set $\mathcal{T}$, set $\mathcal{R}$, set $\mathcal{K}$, set $\overline{\mathcal{S}}_j$

1 **for** *(each $t\in\mathcal{T}$)* **do**
2 $\quad\mathcal{G}_t \leftarrow$ `compute_by_Eq.17(`$\mathcal{J},\mathcal{M},t$`);`
3 $\quad\mathcal{E}_t \leftarrow \emptyset$; $ite \leftarrow 0$;
4 $\quad\mathcal{E}_t \leftarrow$ `subset_bt(`$it, ite, tl, \mathcal{G}_t, \mathcal{R}, \mathcal{K}, \overline{\mathcal{S}}_j, \mathcal{E}_t$`);`
5 $\quad$**if** *($\mathcal{E}_t \neq \emptyset$)* **then**
6 $\quad\quad$**for** *(each $r\in\mathcal{R}$ with capacity c)* **do**
7 $\quad\quad\quad\mathcal{W}_{rt} \leftarrow$ `create_strengthening_MIP(`$r, c, \mathcal{G}_t, \mathcal{E}_t$`);`
8 $\quad\quad\quad\bar{q}_{rjmt} \leftarrow$; `opt(`$\mathcal{W}_{rt}$`);`
9 $\quad\quad\quad$`create_replace_constraints(Ineq.(10),` $\bar{q}_{rjmt}$`, Ineq.(21));`
10 $\quad$**if** *time limit tl is reached* **then**
11 $\quad\quad$**break**;

Let $\mathcal{E}_t = (\bar{e}_1, \bar{e}_2, \ldots, \bar{e}_n)$ be the set of all these feasible subsets. This set can be computed by using a simple backtracking algorithm that recursively proceeds, from level 0 to $|\mathcal{G}_t|$; tentatively fixing the allocation of each respective job and mode to 0 or 1; proceeding to the next level on the search space only when the partial fixation to the current level does not violate any resource or precedence constraint.

If we enumerate all the possibilities over $\mathcal{G}_4$, we could have $2^{13} = 8192$ feasible subsets. However, due to multiple constraints, there are only 51 feasible subsets in $\mathcal{E}_4$:

$$\begin{aligned}\mathcal{E}_4 = [&\{(1,0)\}, \boxed{\{(2,2),(3,0),(1,0)\},}\{(2,2),(1,0)\},\\ &\{(3,0),(1,0)\},\{(1,1)\},\{(2,2),(3,0),(1,1)\},\{(2,2),(1,1)\},\\ &\{(3,0),(1,1)\},\{(1,2)\},\{(2,1),(3,0),(1,2)\},\{(2,1),(1,2)\},\\ &\{(2,2),(3,0),(1,2)\},\{(2,2),(1,2)\},\{(3,0),(1,2)\},\{(2,1)\},\\ &\{(3,0),(5,2),(2,1)\},\{(3,0),(2,1)\},\{(5,2),(2,1)\},\{(2,2)\},\\ &\{(3,0),(5,0),(2,2)\},\{(3,0),(5,1),(2,2)\},\\ &\{(3,0),(5,2),(2,2)\},\{(3,0),(2,2)\},\{(5,0),(2,2)\},\\ &\{(5,1),(2,2)\},\{(5,2),(2,2)\},\{(3,0)\},\{(5,0),(9,1),(3,0)\},\\ &\{(5,0),(3,0)\},\{(5,1),(9,1),(3,0)\},\{(5,1),(3,0)\},\\ &\{(5,2),(9,1),(3,0)\},\{(5,2),(3,0)\},\{(6,1),(3,0)\},\\ &\{(6,2),(9,1),(3,0)\},\{(6,2),(3,0)\},\{(9,1),(3,0)\},\{(5,0)\},\\ &\{(9,1),(5,0)\},\{(9,2),(5,0)\},\{(5,1)\},\{(9,1),(5,1)\},\\ &\{(9,2),(5,1)\},\{(5,2)\},\{(9,1),(5,2)\},\{(6,1)\},\\ &\{(9,2),(6,1)\},\{(6,2)\},\{(9,1),(6,2)\},\{(9,1)\},\{(9,2)\}]\end{aligned}$$

As an example of the impact of considering multiple constraints for reducing the number of valid incidence vectors, if we do not consider precedence constraints and we just consider one renewable resource constraint in the enumeration process, 182 feasible subsets would be built.For the maximal feasible subset `{(2,2),(3,0),(1,0)}`, highlighted above inside the box, if we do not consider all resources and precedence constraints, it will be extended to `{(2,2),(3,0),(5,2),(1,0)}`, `{(2,2),(3,0),(6,2),(1,0)}`.

The ith feasible subset $\bar{e}_i$ contains ordered pairs $(j,m)\in\mathcal{G}_t$. For each renewable resource r with capacity c and time t the following linear program (W_{rt}) can be used to strengthen constraints (10) if the enumeration process is successful, i.e., a pre-defined maximum number of iterations (it) was not reached, (see Algorithm 1, lines 5–9). Consider the continuous variables u_{jm}, indicating the number of consumed units of resource r by job j at mode m in the

strengthened constraint of the following linear programming (W_{rt} model):

Maximize:

$$\sum_{(j,m)\in\mathcal{G}_t} u_{jm} \tag{18}$$

Subject to:

$$\sum_{(j,m)\in\bar{e}} u_{jm} \le c \qquad \forall \bar{e} \in \mathcal{E}_t \tag{19}$$

$$q_{rjm} \le u_{jm} \le c \qquad \forall (j,m) \in \mathcal{G}_t \tag{20}$$

Consider $\forall (r,t) \in \mathcal{R} \times \mathcal{T} : \bar{q}_{rjmt} = u^*_{jm}$ from W_{rt}, where u^*_{jm} is the optimal solution in W_{rt}. This value is introduced as a new input data for the main formulation:

$\bar{q}_{rjmt}$: new values for required amount of renewable resource $r \in \mathcal{R}$ to execute job $j \in \mathcal{J}$ on mode $m \in \mathcal{M}_j$ at time t.

Constraints (21) are created with the new values $\bar{q}_{rjmt}$, yielding improved capacity constraints for renewable resources.

$$\sum_{j\in\mathcal{J}} \sum_{m\in\mathcal{M}_j} \bar{q}_{rjmt} z_{jmt} \le \check{q}_r \qquad \forall r \in \mathcal{R}, \quad \forall t \in \mathcal{T} \tag{21}$$

Due to the bounds on variables u, constraints (21) always dominate the original constraints (10), since $\bar{q}_{rjmt} \ge q_{rjm}$. In particular, whenever $u_{jm} > q_{rjm}$ it dominates strictly. The following defines dominance between two generated cuts Wolsey (1998):

Definition 3.1. Let $c^{1^T} x \le r^1$ and $c^{2^T} x \le r^2$ be two inequalities. We say $c^{1^T} x \le r^1$ dominates $c^{2^T} x \le r^2$ if $c^1_i \ge c^2_i$ $\forall i$ and $r^1 \le r^2$; if at least one of these inequalities is satisfied as an strict inequality, then there is a strict dominance.

An interesting property of this procedure is that it may strengthen constraints of resources that are *not scarce*, given the scarceness of other resources and/or precedence constraints.

Example. By solving the MILP model $\mathcal{W}_{0,4}$ to our example introduced above, the coefficient of the emphasized variable $\boldsymbol{z_{5,2,4}}$ corresponding to job 5 on mode 2 processed at $t = 4$ can be strengthened to 6, without excluding any feasible integer solution. We can generate the following strengthened constraint:

$$0z_{1,0,4} + 0z_{1,1,4} + 0z_{1,2,4} + 0z_{2,1,4} + 0z_{2,2,4} + 3z_{3,0,4} + 0z_{5,0,4} + 0z_{5,1,4} + \mathbf{6z_{5,2,4}} + 0z_{6,1,4} + 6z_{6,2,4} + 0z_{9,1,4} + 9z_{9,2,4} \le 9.$$

For the SMRCPSP and MMRCPSP, the original constraints (10) can be replaced by new constraints presented in (21) in the case that the latter are stronger. For the MMRCMPSP, the new constraints (21) are created to strengthen renewable resources for each project separately, and the original constraints remain in the model.

We consider a time limit *tl*, which will be checked after the subset enumeration procedure for each t. If the time limit is reached, the remaining time periods t will be skipped and the algorithm is terminated (see Algorithm 1, lines 10–11). We continue to find feasible subsets while it does not reach the last element of $\mathcal{G}_t$. If the maximum number of iterations *it* is reached, we stop the process (by returning $\emptyset$, see Algorithm 1, lines 4–5) and continue to the next value of t on the strengthening algorithm.

4. The cutting plane algorithm

The performance of general purpose MILP solvers on a given formulation strongly depends on how tight is the LP relaxation (dual bound) to the optimal solution (Wolsey, 1998). Cutting planes are commonly used to improve this bound by iteratively adding violated cuts.

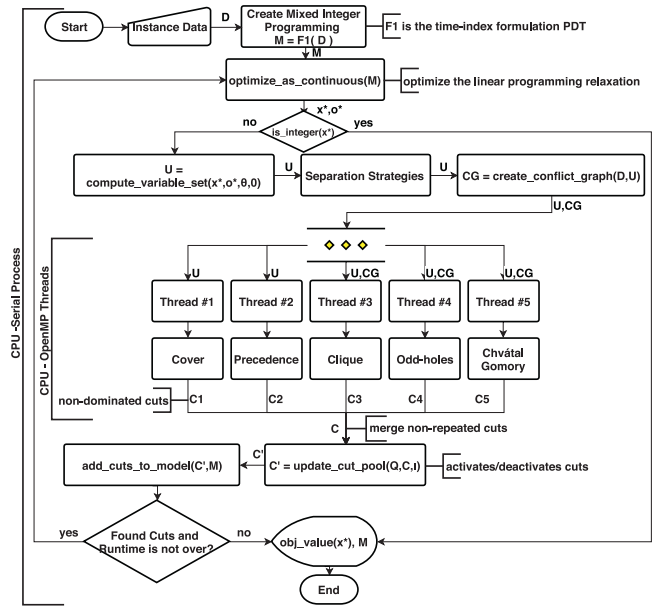


Fig. 2. Execution flow of the proposed cutting plane method.

The proposed cutting plane algorithm uses traditional RCPSP cuts enhanced with new lifting techniques (see Sections 4.1 and 4.2, respectively for cover and precedence cuts), conflict-based cuts (see Section 4.3 for clique and odd-holes cuts) and strengthened Chvátal-Gomory cuts (see Section 4.4) generated from an implicit dense conflict dynamic graph.

An outline of the proposed method is depicted in Fig. 2. After the instance data is read, we create the mathematical programming model (**M**) explained in Section 3 and execute the preprocessing routines. Then, the optimal LP relaxation is computed and if a fractional solution is obtained, different search methods are started to separate violated inequalities. Since the separation of these inequalities may involve the solution of $\mathcal{NP}$-hard problems, we execute the separation procedures in parallel. This allows us to save some processing time in order to process a larger number of iterations in the cutting plane algorithm within the given time limit.

All generated cuts are inserted into a cut pool where repeated inequalities are discarded. Our algorithm quickly discards repeated cuts by using a hash table. While checking for repeated cuts is very fast, the dominance check is slower since it requires checking the contents of the cuts. We only check the dominance in the pool of cover separation since they generated less cuts compared with other types of cuts.

If new cuts have been found after the separation procedure, a stronger formulation is obtained, and the process is repeated. When the time limit is reached or when no further cut is generated, the strengthened model and its objective function value are returned. In the following, we present the different inequalities that are separated as well as the algorithmic aspects involved in their separation.

4.1. Lifted RCPSP knapsack cover cuts - LCV

Resource usage constraints present a knapsack structure which was exploited in Zhu et al. (2006) to generate GUB cover (CV) cuts. Consider the general case of a constraint in the form $\sum_{j\in\mathcal{N}} b_j z_j \le c$, $(b,c) \in \mathbb{N}^{+^n} \times \mathbb{N}^+$ for a set $\mathcal{N}$ of binary variables. The following knapsack problem can be solved to generate a valid inequality that cuts fractional point z^*:

Minimize

$$\gamma(z^*) = \sum_{j\in\mathcal{N}}(1 - z_j^*)v_j \tag{22}$$

subject to:

$$\sum_{j\in\mathcal{N}} b_j v_j > c \tag{23}$$

$$v_j \in \{0, 1\} \qquad \forall j \in \mathcal{N} \tag{24}$$

Whenever a solution with $\gamma(z^*) < 1$ is found involving a set $\mathcal{V} = \{j \in \mathcal{N} : v_j = 1\}$ a violated inequality in the form $\sum_{j\in\mathcal{V}} z_j \le |\mathcal{V}| - 1$ is generated. Since only active variables ($z_j^* > 0$) are considered in this separation, many dominated inequalities can be generated in different iterations.

While general purpose lifting strategies (Gu et al., 2000) can be used, specific problem information can provide an effective procedure to produce lifted cover inequalities. A variable in a traditional cover inequality represents whether a job j and mode m were allocated or not at time t. The proposed lifted RCPSP knapsack cover cut (LCV) separation routine may include, for each job j, variables of additional processing modes without increasing the right-hand-side, producing much stronger cuts. In the single mode case the strengthening will be similar to the one obtained with traditional lifting (see Balas, 1975; Balas and Zemel, 1978; Nemhauser and Vance, 1994) techniques.

The new lifted knapsack cover separation problem for the RCPSP is solved for each renewable resource and each time period. Consider period t^*, resource r with capacity $\breve{q}_r$ and a fractional solution with variable values $z^*_{jmt^*}$. The decision variables are:

- $v_{jm} \in \{0, 1\}$: if variable of job j at mode m is selected (1) or not (0);
- $o_j \in \{0, 1\}$: if at least one mode for job j is included in the cut (1) or not (0);
- $\underline{w}_{jm} \in \{0, 1\}$: if job j has m as the selected mode with the smallest resource consumption from the selected ones (1) or not (0);
- $\underline{e} \in \mathbb{Z}^+$: resource consumption excess if modes with smallest resource consumption are selected.
- $\overline{v} \in \mathbb{R}^+$: cut violation.

The LCV separation problem is given by:

Maximize

$$\omega\overline{v} + \sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}_j} \mu v_{jm} \tag{25}$$

subject to:

$$o_j = \sum_{m\in\mathcal{M}_j} \underline{w}_{jm} \qquad \forall j \in J \tag{26}$$

$$\underline{e} = \sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}_j} q_{rjm}\underline{w}_{jm} - \breve{q}_r \tag{27}$$

$$v_{jm} \le \sum_{m'\in\mathcal{M}_j : q_{rjm'} \le q_{rjm}} \underline{w}_{jm'} \qquad \forall j \in \mathcal{J}, m \in \mathcal{M}_j \tag{28}$$

$$\overline{v} = \sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}} z^*_{jmt^*} v_{jm} - \sum_{j\in\mathcal{J}} o_j + 1 \tag{29}$$

$$0.005 \le \overline{v} \le \infty \tag{30}$$

$$1 \le \underline{e} \le \infty \tag{31}$$

$$o_j, v_{jm}, \underline{w}_{jm} \in \{0, 1\} \qquad \forall j \in \mathcal{J}, \quad \forall m \in \mathcal{M}_j \tag{32}$$

The objective function (25) maximizes a hierarchical objective function composed of, first, the cut violation and, second, the inclusion of additional jobs and modes in the generated inequality to produce stronger cuts ($\omega \gg \mu$). These weights cannot be too large or too small to prevent numerical instability in the solvers. Constraints (26) ensure that o_j is only activated when some mode is selected for job j as the mode with lower resource usage. Constraints (27) and (31) ensure that a cover is produced. Constraints (28) ensure that only modes with resource usage greater than or equal to the mode with the smallest resource usage selected are allowed for lifting. Equality (29) computes the cut violation and constraint (30) ensures that a violated cut is produced. Finally, constraints (32) ensure that variables o_j, v_{jm} and $\underline{w}_{jm}$ can only assume binary values.

Let C be the set of selected jobs and modes (with $v_{jm} = 1$) in the solution of the problem above. Further, let o_j indicate its corresponding solution values. We may generate they following LCV cut:

$$\sum_{(j,m)\in C} z_{jmt} \le \sum_{j\in J} o_j - 1 \qquad \forall t \in \mathcal{T} \tag{33}$$

For a valid cover cut the value on the right-hand-side would be the size of the set minus 1, with the lifting strategy whenever it chooses more than one mode per job the sum of the right-hand-side will be strictly smaller than the size of the set minus 1, generating a stronger cut as in the example below.

Example. Consider the following cut for resource r_0 on $t = 8$, and jobs {4,8} processing respectively on modes {0,2} from Fig. 1, generated using the separation described in (22)-(24):

$$(CV) = z_{4,0,8} + z_{8,2,8} \le 1.$$

This cut can be lifted since job 8 has other modes {0,1}, each of them consuming 6 units of resource r_0, more than the current mode {2} that consumes 3 units, forming still a valid inequality:

$$(LCV) = z_{4,0,8} + \boldsymbol{z_{8,0,8} + z_{8,1,8}} + z_{8,2,8} \le 1.$$

It is important to emphasize that the first component of the objective function maximizes the violation considering the consumption of the other modes, and the second component, even for the single mode version, will add additional variables that do not contribute to the cut violation but which contribute to generate a stronger inequality.

4.2. Lifted precedence based cuts - LPR

In addition to the cover cut introduced above, it is possible to further strengthen the formulation by analyzing the precedence between jobs and using precedence (PR) cuts similar to those used by Zhu et al. (2006). Consider a job j, its (direct or indirect) successor s and a time t. Also, consider the following constants $e_j = \min_{m\in\mathcal{M}_j}(\mathcal{T}_{jm})$ and $l_j = \max_{m\in\mathcal{M}_j}(\mathcal{T}_{jm})$ to limit the time period. We introduce a new lifted version for the precedence cuts, shown in Eq. (34). Consider the lengths of the shortest paths on the precedence graph, $\breve{d}_{jms}$ and $\breve{d}^*_{js}$, introduced in Section 3.2. The following inequalities are valid:

$$\sum_{m\in\mathcal{M}_j}\sum_{t'=e_j}^{t+\breve{d}^*_{js}-\breve{d}_{jms}} x^*_{jmt'} \ge \sum_{m\in\mathcal{M}_s}\sum_{t'=e_s}^{min(l_s, t+\breve{d}^*_{js})} x^*_{smt'}$$
$$\forall j \in \mathcal{J}, s \in \overline{\mathcal{S}}_j, t \in \{\max(e_j, e_s - \breve{d}^*_{js}), \ldots, min(l_j, l_s - \breve{d}^*_{js})\} \tag{34}$$

Example. Consider the following two cuts. The first one, (PR), was generated with the RCPSP precedence cut as proposed in Zhu et al. (2006). The second one, (LPR), is the lifted inequality (34) to predecessor job 5 and its successor 10 at time period 9:

$$\begin{aligned}(PR) = & -x_{5,0,2} - x_{5,0,3} - x_{5,0,4} - x_{5,0,5} - x_{5,0,6} - x_{5,0,7} - x_{5,0,8} \\ & -x_{5,0,9} - \boxed{x_{5,0,10} - x_{5,0,11}} - x_{5,1,2} - x_{5,1,3} - x_{5,1,4} - x_{5,1,5} \\ & -x_{5,1,6} - x_{5,1,7} - x_{5,1,8} - \boxed{x_{5,1,9} - x_{5,1,10} - x_{5,1,11}} - x_{5,2,2} \\ & -x_{5,2,3} - x_{5,2,4} - x_{5,2,5} - x_{5,2,6} - \boxed{x_{5,2,7} - x_{5,2,8} - x_{5,2,9}} \\ & +x_{10,0,11} + x_{10,2,11} \leq 0; \\ (LPR) = & -x_{5,0,2} - x_{5,0,3} - x_{5,0,4} - x_{5,0,5} - x_{5,0,6} - x_{5,0,7} - x_{5,0,8} \\ & -x_{5,0,9} - x_{5,1,2} - x_{5,1,3} - x_{5,1,4} - x_{5,1,5} - x_{5,1,6} - x_{5,1,7} \\ & -x_{5,1,8} - x_{5,2,2} - x_{5,2,3} - x_{5,2,4} - x_{5,2,5} - x_{5,2,6} + x_{10,0,11} \\ & +x_{10,2,11} \leq 0.\end{aligned}$$

Notice the change in the coefficients of the variables on the left-hand-side of the lifted cut (LPR), based on Eq. (34) and corresponding to the variables of job 5. In the case of different durations for processing modes of a job j, there will be an increase in the sum of the coefficients of the left-hand-side of (LPR) since we use $\check{d}_{jms}$ instead of just considering the fastest mode given by $\check{d}^*_{js}$ (as proposed in Zhu et al., 2006). The original cut variant (PR) from Zhu et al. (2006) considers the fastest time. The variables highlighted in the boxes in the (PR) cut are not included in our cut approach (LPR). The lifted cut therefore strictly dominates the original cut, given that the highlighted variables have coefficients 0 (instead of −1) in the cut stated as $\leq$ inequality.

Our separation procedure (see Algorithm 2) therefore selects different paths (line 2) that connect job j and the artificial project completion job a_p. In particular, it is desirable to avoid redundant constraints, i.e., constraints of predecessor/successor jobs belonging to the same path in the dependency graph with a similar meaning, for jobs on subpaths that have already been used in previously added precedence cuts. Whenever a violated precedence cut is found on this path (lines 8–13), the remaining jobs in this path are skipped (line 15). We limit the number ζ of precedence cuts that can be added per round and only add the most violated cuts (line 18).

Algorithm 2: Lifted_precedence_based_cuts.

Data: set of jobs $\mathcal{J}$, artificial project completion job a_p, maximum number of cuts ζ

1 **for** *(each* $j \in \mathcal{J}$*)* **do**
2 $\quad \check{P} \leftarrow$ `compute_paths`(j, a_p);
3 $\quad pos \leftarrow 0$;
4 $\quad$ **for** *(each* $\check{p} \in \check{P}$*)* **do**
5 $\quad\quad s \leftarrow$ `successor`$(j, \check{p}, pos)$;
6 $\quad\quad found \leftarrow 0$;
7 $\quad\quad$ **for** $(t \in \{\max(e_j, e_s - \check{d}^*_{js}), \ldots, \min(l_j, l_s - \check{d}^*_{js})\})$ **do**
8 $\quad\quad\quad$ $\mathbf{c} \leftarrow$ `init_cut`$()$;
9 $\quad\quad\quad$ $(V, C) \leftarrow$identifies a set of variables and their coefficients that generate a violated precedence cut for the predecessor j and successor s at time period t;
10 $\quad\quad\quad$ **if** $(V \neq \emptyset)$ **then**
11 $\quad\quad\quad\quad$ `add_set_var`$(\mathbf{c}, V, C)$;
12 $\quad\quad\quad\quad$ $C \leftarrow C \cup \{$`cut`$(\mathbf{c}, \geq, 0)\}$;
13 $\quad\quad\quad\quad$ $found \leftarrow 1$;
14 $\quad\quad$ **if** *(found)* **then**
15 $\quad\quad\quad$ skipp the remaing jobs in the path of j to a_p;
16 $\quad\quad$ **else**
17 $\quad\quad\quad$ $pos \leftarrow pos + 1$;
18 `sort_cuts_by_highest_violation_and_filters`$(\mathcal{C}, \zeta)$;
19 `return` $\mathcal{C}$;

4.3. Conflict-based cuts: cliques (CL) and odd-holes (OH)

According to Padberg (1973), LP relaxations for problems that mostly contain binary variables linked to generalized upper bound (GUB) constraints can be significantly strengthened by the inclusion of inequalities derived from the set packing polytope (SPP). Generally, clique and odd-holes cuts can be generated using Conflict Graphs ($\mathcal{CG}$). The denser the $\mathcal{CG}$, the more inequalities can be generated. The disadvantage of having dense $\mathcal{CG}$ is that they can be prohibitively large (Atamtürk et al., 2000), so our algorithm creates the $\mathcal{CG}$ dynamically at each iteration by considering a set ($\mathcal{U}$) that contains the variables of interest, i.e., variables that have a non-zero value in the LP relaxation or variables set to zero but with a small reduced cost.

Several well-known inequalities can be generated considering pairwise conflicts between binary variables stored in $\mathcal{CG}$. Some conflicts can be easily detected by solvers considering constraints such as (8). Other conflicts can be implied from optimality conditions or by analyzing problem specific structures. Overall, the denser is the $\mathcal{CG}$, stronger are the produced cuts.

The dynamic dense $\mathcal{CG}$ created is used in a separation procedure for inequalities derived from a common class of cuts for the SPP: cliques and odd-holes. A clique inequality for a set C of conflicting variables has the form $\Sigma_{j \in C} x_j \leq 1$ and an odd-hole inequality for a cycle C can be defined as: $\sum_{j \in C} x_j \leq \lfloor \frac{|C|}{2} \rfloor$ Santos et al. (2016).

Santos et al. (2016) present a clique separation routine that separates all violated cliques into a conflict subgraph induced by fractional variables. The authors then present a lifting that extends generated cliques considering the original $\mathcal{CG}$. They also present a strengthening of odd-holes inequalities by the inclusion of a so-called wheel center. For an odd-hole with variables C and W being the set of candidates to be included as wheel centers of C, the inequality (35) is valid:

$$\sum_{j \in W} \lfloor \frac{|C|}{2} \rfloor x_j + \sum_{j \in C} x_j \leq \lfloor \frac{|C|}{2} \rfloor \tag{35}$$

Our approach presented in Algorithm 3 considers four conflict types for variables x_{jmt}:

1. conflicts between variables of the same job (lines 6–7);
2. conflicts involving jobs that if allocated at the same time exceed the capacity of available renewable resources (lines 8–10);
3. conflicts based on precedence relations, in which the time window between the predecessor job j on mode m and some $s \in \overline{\mathcal{S}}_j$ is smaller than $\check{d}_{jms}$ (lines 11–14);
4. conflicts considering jobs of different projects, where the sum of the delays generated by allocating these jobs in specific positions implies a total delay greater than α (lines 15–16).

After the creation of the $\mathcal{CG}$, cuts can be generated considering the current fractional solution. In this paper, we use the routines described in Brito et al. (2015), where cliques and odd-holes are exactly separated and lifted.

Example. Considering the following clique cut:

$$(CL) = x_{5,0,12} + x_{5,1,11} + x_{5,2,9} + x_{10,0,11} + x_{10,0,12} + x_{10,0,13} + x_{10,2,11} + x_{10,2,12} + x_{10,2,13} \leq 1.$$

Algorithm 3: creating_conflict_graph.

Data: variables set $\mathcal{U}$, set $\mathcal{R}$, set $\bar{\mathcal{S}}$, delay α
Result: Conflict Graph $\mathcal{CG}$

```
CG ← ∅;
for (v1 ∈ U) do
    j1 ← job(v1); m1 ← mode(v1); t1 ← time(v1);
    for (v2 ∈ U) do
        j2 ← job(v2); m2 ← mode(v2); t2 ← time(v2);
        if (j1 = j2) then
            add_var_conf(CG, v1, v2); continue;
        if (t1 = t2) then
            for (r ∈ R : (q_{r j1 m1} + q_{r j2 m2} > q̆_r)) do
                add_var_conf(CG, v1, v2); continue;
        if (j2 ∈ S̄_{j1}) then
            w ← d*_{j1,j2} − job_min_dur(j1);
            if ((end_time(j1) > t2) or (t2 − end_time(j1) < w))
            then
                add_var_conf(CG, v1, v2); continue;
        if (proj(j1) != proj(j2) & (delay(j1, m1, t1) +
        delay(j2, m2, t2)) > α) then
            add_var_conf(CG, v1, v2);
return CG;
```

It is possible to observe, from Fig. 1, that the variables corresponding to jobs 5 and 10 have conflicts at different time periods considering different modes, given that they have a precedence relation.

Example. Consider the following odd-hole cut:

$(OH) = x_{1,0,0} + x_{1,0,5} + x_{2,1,0} + x_{2,1,3} + x_{5,0,3} \leq 2.$

Still referring to job 5 in Fig. 1, we can observe conflicts from the precedence relationship with job 1. Also, a low amount of resources available at times when jobs 5 and 2 intersect on the variables of the example above reflect conflicts between them. Thus, these three jobs can not be allocated in parallel, considering the amount of resources consumed by their modes.

4.4. Strengthened Chvátal-Gomory cuts - (SCG)

Chvátal-Gomory (CG) cuts are well-known cutting planes for MILP models. The inclusion of these cuts allows to significantly reduce the integrality gaps, even when only rank-one cuts are employed, i.e., those obtained from original problem constraints (Fischetti and Lodi, 2007).

Consider the integer linear programming (ILP) problem as $\min\{\mathbf{c}^T\mathbf{x}$: $A\mathbf{x} \leq \mathbf{b}$, $\mathbf{x} \geq 0$ integer$\}$, where $A \in \mathbb{R}^{mxn}$, $\mathbf{b} \in \mathbb{R}^m$, and $\mathbf{c} \in \mathbb{R}^n$, with the two associated polyhedra $P := \{\mathbf{x} \in \mathbb{R}^n_+ : A\mathbf{x} \leq \mathbf{b}\}$ and $P_{\mathcal{I}} := conv\{\mathbf{x} \in \mathbb{Z}^n_+ : A\mathbf{x} \leq \mathbf{b}\} = conv(P \cap \mathbb{Z}^n)$ with $\mathbf{x}$ being integer variables. Consider $\mathcal{I}$ and $\mathcal{H}$ the sets of constraints and variables, respectively.

A Chvátal-Gomory cut Chvátal (1973) is defined as a valid inequality for $P_{\mathcal{I}}$: $\lfloor \mathbf{u}^T A \rfloor \mathbf{x} \leq \lfloor \mathbf{u}^T \mathbf{b} \rfloor$, where $\mathbf{u} \in \mathbb{R}^m_+$ is a multiplier vector. The choice of $\mathbf{u} \in \mathbb{R}^+$ is crucial to deriving useful inequalities. Fischetti and Lodi (2007) propose the MILP model for Chvátal-Gomory separation. The maximally violated $\sum_{j \in \mathcal{H}(x^*)} a_j x_j \leq a_0$ inequality can be found by optimizing the following separation MILP model:

Maximize:

$$\sum_{j \in \mathcal{H}(x^*)} a_j x^*_j - a_0 \tag{36}$$

subject to:

$$f_j = \mathbf{u}^T A_j - a_j, \qquad \forall j \in \mathcal{H}(x*) \tag{37}$$

$$f_0 = \mathbf{u}^T \mathbf{b} - a_0 \tag{38}$$

$$0 \leq f_j \leq 1 - \delta \qquad \forall j \in \mathcal{H}(x*) \cup \{0\} \tag{39}$$

$$-1 + \delta \leq u_i \leq 1 - \delta \qquad \forall i = 1, \ldots, m \tag{40}$$

$$a_j \in \mathbb{Z}, \qquad \forall j \in \mathcal{H}(x*) \tag{41}$$

where $\mathcal{H}(x^*) := \{j \in 1, \ldots, \check{n} : x^*_j > 0\}$ and x^* are fractional values for all $\check{n}$ variables of an LP solution for a general problem fixed in the MILP. To strengthen the cut, a penalty term $\Sigma_i w_i u_i$, with $w_i = 10^{-4}$ for all i, is applied to the objective function. To improve the numerical accuracy of the method, multipliers too close to 1 are forbidden ($u_i \leq 0.99$, $\forall i$).

Example. Consider the following cut generated with the CG for jobs from Fig. 1:

$$(CG) = 3x_{1,0,8} + x_{5,0,5} + 2x_{5,0,6} + 4x_{6,1,4} + 2x_{6,1,8} + 2x_{8,1,7} + 2x_{9,1,7} \leq 5.$$

On the one hand, the larger the set of non-redundant and tight constraints considered in the Chvátal-Gomory separation the more likely it is that violated inequalities will be found. On the other hand, large separation problems can be hard to solve and the overall performance of the cutting plane method can degrade. The following subsection will therefore consider specific strategies to find suitable sets of constraints.

4.4.1. Finding a set of constraints

In our approach we consider a tuple $(\bar{s}, \bar{f})$ to indicate the starting time and the finishing time of a given interval with size η to filter the constraints and variables sets. We compute, for different intervals $(\bar{s}, \bar{f})$, the summation of all infeasibilities for the integrality constraints for all their variables x_{jmt} where $\bar{s} \leq t \leq \bar{f}$, x^*_{jmt} are the fractional values for the RCPSP variables. The value $\hat{f}$ is composed of the sum of the nearest integer distance, to indicate how fractional the variables in that interval are.

$$\hat{f}_{(\bar{s},\bar{f})} = \sum_{j \in \mathcal{J}} \sum_{m \in \mathcal{M}_j} \sum_{t=\bar{s}}^{\bar{f}} |round(x^*_{jmt}) - x^*_{jmt}| \qquad \forall\ (\bar{s}, \bar{f}) \in \mathcal{T} \tag{42}$$

To find the most fractional interval $\hat{f}_{(\bar{s},\bar{f})}$ in the scheduling, in which the constraints with their respective variables will be chosen, we start from the beginning of the scheduling, and we slide the interval $(\bar{s}, \bar{f})$ until the end of the scheduling (see Algorithm 4, line 2). The parameters on the algorithm indicate the interval size η, the jump size ι to go to the next interval and a percentage ζ that allows sliding the interval. The slide only occurs if the violation is greater than the current value, plus the percentage of ζ. Another input data is the dynamic conflict graph $\mathcal{CG}$.

Preliminary experiments showed that cuts separated using constraints from the beginning of the time horizon were more effective for improving the dual bound, jobs allocated in the first time period are responsible for pushing the allocations of the others in the precedence graph. It is therefore desirable to find integer values for the first ones. Once the most fractional interval is found, the most important constraints are identified to compose the set for the Chvátal-Gomory separation.

The renewable resources directly impact the duration of the projects; therefore, all the variables of these constraints into the interval $(\bar{s}, \bar{f})$ are considered in the set $\mathcal{V}$ (see Algorithm 4, lines 3–7). Further constraints from the original problem are included in the separation problem whenever they are related to the current

Algorithm 4: Finding_set_constraints.

Data: fractional RCPSP solution x^*, rows set $\mathcal{I}$, conflict graph $\mathcal{CG}$, size interval η, size jump ι, percentage allowed to slide the interval ζ

Result: Set $\mathcal{Z}$ of selected rows

1 $\mathcal{V} \leftarrow \emptyset$;
2 $(\bar{s}, \bar{f}) \leftarrow$ `identify_interval`$(x^*, \eta, \iota, \zeta)$;
3 **for** $((r_1 \in \mathcal{I})$ *and* `is_in_interval`$(r_1, \bar{s}, \bar{f}))$ **do**
4 $\quad$ *type* $\leftarrow$ `type_row`(r_1);
5 $\quad$ **if** $(type = Ineq.10)$ **then**
6 $\quad\quad$ `add_row`$(\mathcal{Z}, r_1)$;
7 $\quad\quad$ $\mathcal{V} \leftarrow \mathcal{V} \cup$ `vars_row`(r_1);
8 $O \leftarrow$ `jobs`$(\mathcal{V})$;
9 **for** $(j \in O)$ **do**
10 $\quad$ $\mathcal{V}_j \leftarrow$ `vars_jobs`$(\mathcal{V}, j)$;
11 $\quad$ $r_2 \leftarrow$ `create_constraints`$(\leq, 1)$;
12 $\quad$ `add_set_var`$(r_2, \mathcal{V}_j)$;`add_row`$(\mathcal{Z}, r_2)$;
13 $\mathcal{K} \leftarrow$ `nonrenewable_resources`$(\mathcal{V})$;
14 **for** $(k \in \mathcal{K})$ **do**
15 $\quad$ $\mathcal{V}_k \leftarrow$ `vars_nonrenewable_resources`$(\mathcal{V}, k)$;
16 $\quad$ $r_3 \leftarrow$ `create_constraints`$(\leq, \text{capacity}(k))$;
17 $\quad$ `add_set_var`$(r_3, \mathcal{V}_k)$;`add_row`$(\mathcal{Z}, r_3)$;
18 $\mathcal{C} \leftarrow$ `pairs_of_conflicting_variables`$(\mathcal{CG}, \mathcal{V})$;
19 **for** $(c \in \mathcal{C})$ **do**
20 $\quad$ $\mathcal{V}_c \leftarrow$ `vars_conflicts`$(\mathcal{V}, c)$;
21 $\quad$ $r_4 \leftarrow$ `create_constraints`$(\leq, 1)$;
22 $\quad$ `add_set_var`$(r_4, \mathcal{V}_c)$;`add_row`$(\mathcal{Z}, r_4)$;
23 **return** $\mathcal{Z}$;

time window: constraints that restrict the choice of only job (lines 8–12) and non-renewable resource constraints (lines 13–17). Functions `nonrenewable_resources`$(\mathcal{V})$ and `jobs`$(\mathcal{V})$ returns, respectively, the set of non-renewable resources and jobs where variables of $\mathcal{V}$ appear. A good strategy is to find additional constraints that represent conflicts between the variables of set $\mathcal{V}$. A main conflict is analyzed, whereby the time window comprising variables coming from the conflict graph $\mathcal{CG}$ generated dynamically at each iteration of the cutting plane received as input parameter (lines 18–22). Function `pairs_of_conflicting_variables`$(\mathcal{CG}, \mathcal{V})$ returns the pairs of these conflicting variables.

4.4.2. Strengthening procedure

To produce strengthened CG cuts, a strategy similar to the proposal of Letchford et al. (2016) is employed. The key idea is to take violated CG cuts and then strengthen the right-hand-sides (*rhs*). Letchford et al. (2016) solves the maximum weight stable set problem for the conflict graph induced by the binary variables of the CG cut to find a (hopefully smaller) new valid *rhs*. Our approach solves the same problem augmented by additional constraints involving these variables. Consider here that set $\mathcal{H}$ contains all variables of the cut to be strengthened. $A_{\mathcal{H}}$ is the matrix of coefficients of $\mathcal{H}$ including non-renewable and renewable resource constraints, job allocation constraints and conflict constraints, with *rhs* values specified in a vector $\mathbf{b}$. The vector $\mathbf{c}$ are the coefficients of the variables that appear in the cut. Consider vector $\mathbf{x}$ as variables of $\mathcal{H}$ and the integer linear programming as $\max\{\mathbf{c}^T\mathbf{x} : A_{\mathcal{H}}\mathbf{x} \leq \mathbf{b}\}$. If the optimal solution value of this MILP is smaller than the original *rhs* of the cut, the original CG can be strengthened with this new value on the *rhs*.

Example. Considering the previous example, it is possible to strengthen the Chvátal-Gomory cut by tightening the value on the right side to 4 based on the MILP presented before. Notice that by applying the strengthening, it was possible to reduce the rhs value by 20%:

$$(SCG) = 3x_{1,0,8} + x_{5,0,5} + 2x_{5,0,6} + 4x_{6,1,4} + 2x_{6,1,8} + 2x_{8,1,7} + 2x_{9,1,7} \leq \mathbf{4}.$$

Table 1
Benckmark datasets.

library	variant/num. inst.	group	in dataset	open
PSPLIB	SMRCPSP/2040	J60	79	79
		J90	105	105
		J120	514	514
	MMRCPSP/3931	J30	641	31
MISTA	MMRCMPSP/30	A	10	7
MMLIB	MMRCPSP/4300	J50	540	118
		J100	540	176
		J50+	1620	1178
		J100+	1620	1370

5. Computational results

This section presents the results of the experiments obtained by the proposed cutting plane algorithm and the preprocessing routine to strengthen resource-related constraints. All computational experiments have been carried out on a computing cluster (Compute Canada) composed by Intel ® Xeon X5650 Westmere processors with 2,67 GHz and 512 GB of RAM running Scientific Linux release 6.3. All algorithms were coded in `ANSI C 99` and compiled with GCC version 5.4.0, with flags *-Ofast* and solver GUROBI version 8.0.1 (Gurobi Optimization, 2016).

5.1. Benchmark datasets

In order to facilitate the comparison of algorithmic improvements, several public databases were established, including realistic, but computationally challenging problem instances. In 1996 a public and well-known repository of benchmark datasets, devoted to this class of problems, was established: the Project Scheduling Problem Library - PSPLIB (Kolisch and Sprecher, 1996). In 2013 a new dataset based on PSPLIB instances, considering multiple projects, emerged from the MISTA Challenge (Wauters et al., 2016). Then, in 2014, another dataset based on PSPLIB was proposed for the multi-mode scheduling problem called MMLIB (Van Peteghem and Vanhoucke, 2014). For all problem variants, many of those instances are still open. In particular, by the time of writing this paper, we found that 215 of the PSPLIB instances, 2842 of the MMLIB instances, and 27 of the MISTA instances, were still open.[2]

We now define the benchmark used in this work based on (Schnell and Hartl, 2017; Toffolo et al., 2016; Zhu et al., 2006) and on instances from the PSPLIB, MISTA and MMLIB repositories. From PSBLIB, we consider instances for the SMRCPSP and MMRCPSP problem variants. From MISTA, we included all problem instances from the basic dataset A for the MMRCMPSP. For MMRCPSP, we also use all instances from MMLIB in the final experiments.

Table 1 shows, for each library, the total number of problem instances defined for each problem variant, the group name, the number of instances that are used in our benchmark datasets, and the number of instances for which optimality has not yet been proven in the literature.

[2] 1339 considering the PSPLIB website and 30 considering the MISTA website. There are no informations about the optimal solutions achieved by Schnell and Hartl (2017) and Toffolo et al. (2016).

Table 2
Average integrality gaps and the average computing times (sec.) for solving with the original LP relaxations (LR) and the strengthened LR (SLR).

		LR		SLR	
group	n	gap	time	gap	time
A	10	0.441	85.9	**0.438**	683.6
J30	245	0.663	0.3	**0.658**	2.8
J60	57	0.825	1.0	**0.816**	5.3
J90	80	0.822	2.1	**0.818**	26.9
J120	390	0.840	3.3	**0.835**	37.2
total	782	0.718	18.5	**0.713**	151.2

5.2. Cutting plane algorithm experiments

In order to evaluate the performance of the proposed cutting plane in relation to the specific problem cuts and the linear relaxation, preliminary experiments have been performed on the benchmark datasets with $\alpha > 0$[3] totaling 782 instances from PSPLIB and MISTA. These two sets include instances of the versions SMRCPSP, MMRCPSP and MMRCMPSP. Instances from MMLIB, that contemplate a larger number of varied instances for the MMRCPSP version, were introduced only in experiments carried out using the complete version of our approach. The impact of adding the cuts for instances where $\alpha = 0$ could not be measured, since the LP bound remains at the same value even when the formulation has been improved. We used a time limit of 24 h for each instance from the MMRCMPSP and 4 h for each instance from the SMRCPSP and MMRCPSP.

The first experiment was conducted to evaluate the preprocessing MILP formulation by analyzing the LP relaxation (LR) and the strengthened LP relaxation (SLR) using the coefficient strengthening MILP presented in Section 3.4 to renewable resources constraints. The reported computing time for the SLR includes the time spent to find the successful feasible sets with parameters to stop the enumeration process as $it = 200,000$ and tl as the maximum allowed time defined above to run the approach. Table 2 shows the average integrality gaps[4] and the average computing times in seconds that have been obtained with the original LP relaxations and the strengthened LR for the instances from PSPLIB and MISTA.

The results in Table 2, shown in bold numbers, indicate that the strengthening formulation slightly improves the integrality gaps. As expected, solving only the weak initial formulation is faster than solving the formulation with the preprocessing routines. However, we note that even small improvements in the root node can result in a large number of nodes pruned later in the search tree. For this reason, and given that the additional computing times are still quite reasonable, we use this strengthening strategy in all further experiments.

According to Kolisch (1995), network precedence relationships and the factor between availability-consumption of resources are the two critical characteristics of the instances. When pruning is applied considering these two characteristics, we note that even for instances with 120 jobs, there are time periods for which it is possible to enumerate the feasible subsets. For example, it is possible to enumerate until $t = 102$ for the large instance $j1201_1$. This instance is composed of 120 jobs and has restricted relationship between the availability and consumption of renewable resources.

5.2.1. Results for different cut families

In order to devise an effective cutting plane strategy, we next evaluate the different separation strategies. In Section 5.2, it has been shown that using the SLR instead of the original LR strengthens the formulation while only marginally increasing computing times. We now explore the bound improvement when combining the SLR with each of the different cut types. Tables 3 and 4 show the average integrality gaps and the average computing times for different approaches. In both tables, the first column SLR presents results obtained by the strengthened LP relaxation without cuts. The LCV[5], LPR, CL, OH ad SCG columns indicate, respectively, results[6] obtained by combining the SLR with one additional cut type: lift operations to cover and precedence cuts, clique cuts, odd-hole cuts and strengthened Chvátal-Gomory cuts.

The results in Table 3 suggest that the best average values (in bold numbers) of the integrality gaps were obtained by adding the lifted precedence cuts. Those cuts offered the best bound improvement (about 15%), while requiring only moderate additional computing time. For the complete cutting plane, in the final experiment, we execute the separation procedures in parallel, but if a hierarchical implementation approach was used, one would add the cut families based the order of their integrality gap improvements: LPR, CL, LCV, SCG and OH.

The results of instances from MMRCMPSP are presented in Table 4. The results summarized indicate that the LPR cuts are also effective for the multi-project problem variant, even for large instances. SCG cuts have been found to be particularly useful for this class of problem. Experiments for instances as from A-7 typically exceeded the given time or memory limits to insertion of cuts such as CL, OH, and SCG.

In Table 5 we evaluate the lifting strategies of the traditional RCPSP cuts proposed by Zhu et al. (2006) and the strengthening strategy for Chvátal-Gomory cuts, also combining with the SLR.

The lifting strategies were able to improve the average integrality gaps of all benchmark datasets when comparing with the traditional RCPSP cuts (CV and PR). The results in bold numbers suggest that the lifting strategy is generally successful, substantially improving the average integrality gaps while increasing computing times. Note that even for the benckmark datasets of SMRCPSP, improved lower bounds were achieved, since lifting tends to use more variables in the cut apart from those that contribute to the cover violation. The strengthened CG cuts did not improve upon the original CG cuts, which can be explained by the large computing times spent by the strengthening procedure. When analyzing the average number of iterations for both procedures, the CG without lift did 3118 iterations in the average, while SCG did just 2450 iterations. Even with a reduced number (21%) of iterations it was able to achieve basically the same results.

To analyze the cut generation for each separation strategy, we have computed the number of unique cuts for each strategy to the previous experiment. Fig. 3 shows boxplots of the number of cuts for different instance types and benchmarks.

The lifted precedence strategy LPR finds a reasonable number of cuts for almost all benchmark datasets. The strengthened Chvátal-Gomory SCG strategy seems to find cuts for all the benchmark datasets, including J30 and J90 in which LPR finds less. Another peculiarity of SCG is that although it finds few cuts at each iteration it remains to find cuts for a larger number of iterations. It is also possible to observe that a small number of odd-holes cuts are generated, which explains the little impact on the integrality gaps.

[3] $\alpha > 0$ is an upper bound to the maximum TPD with value higher then its CPD.

[4] given the best known upper bound $\overline{b}$, from PSPLIB and MISTA websites, and an obtained dual bound $\underline{b}$, the integrality gap is computed as follows: $\frac{(\overline{b}-\underline{b})}{\overline{b}}$, $\overline{b} > 0$

[5] $\omega = 100000$ and $\mu = 0.1$

[6] the maximum number of precedence and clique cuts added to the LP at each iteration corresponds to 20% of the amount of the LP rows.

Table 3
Average integrality gaps and average computing times (sec.) obtained by different cuts for the SMRCPSP and MMRCPSP benckmark datasets with the time limit of 4 h.

		SLR		+{LCV}		+{LPR}		+{CL}		+{OH}		+{SCG}	
group	*n*	gap	time	gap	time	gap	time	gap	time	gap	time	gap	time
J30	245	0.658	3	0.644	6	**0.549**	5	0.567	101	0.657	3	0.551	14200
J60	57	0.816	5	0.814	11	**0.712**	88	0.806	1093	0.816	12	0.812	14401
J90	80	0.818	27	0.816	45	**0.672**	454	0.807	3251	0.818	35	0.815	14400
J120	390	0.835	37	0.820	114	**0.714**	2111	0.817	7265	0.835	56	0.832	14423
total	772	0.781	18.1	0.774	43.9	**0.661**	665	0.749	2928	0.781	27	0.780	14408

Table 4
Integrality gaps and computing times (sec.) obtained by different cuts to the MMRCMPSP benckmark datasets with the time limit of 24 h.

	SLR		+{LCV}		+{LPR}		+{CL}		+{OH}		+{SCG}	
inst.	gap	time	gap	time	gap	time	gap	time	gap	time	gap	time
A-1	1.000	0	1.000	0	0.875	0	0.875	0	1.000	0	**0.000**	1793
A-2	1.000	1	1.000	4	0.987	4	0.989	2	1.000	2	**0.935**	86007
A-3	**0.000**	8	**0.000**	9	**0.000**	15	**0.000**	15	**0.000**	15	**0.000**	62097
A-4	0.414	4	0.411	35	**0.333**	39	0.366	1297	0.414	10	0.396	86005
A-5	0.305	32	0.299	1743	**0.273**	1168	0.301	6157	0.305	254	0.302	86028
A-6	0.428	596	0.427	4118	**0.317**	3303	0.413	34106	0.428	1724	0.424	86005
total	0.5245	221	0.523	985	**0.464**	755	0,491	6930	0.525	334	**0.343**	40983

Table 5
Average integrality gaps and average computing times (sec.) obtained by lifting the traditional RCPSP cuts, by strengthening the CG cuts and their original versions combining with the SLR with time limit of 24 h for A group and 4 h for the others.

		+{CV}		+{LCV}		+{PR}		+{LPR}		+{CG}		+{SCG}	
group	*n*	gap	time	gap	time	gap	time	gap	time	gap	time	gap	time
A	6	0.526	59.9	**0.523**	984.8	0.518	555.9	**0.464**	754.8	**0.342**	71825	0.343	67989
J30	245	0.660	0.7	**0.644**	5.6	0.653	3.7	**0.549**	4.9	**0.548**	14092	0.551	14200
J60	57	0.825	2.1	**0.814**	11.2	0.794	30.1	**0.712**	87.5	**0.811**	14402	0.812	14401
J90	80	0.821	4.3	**0.816**	44.9	0.780	131.8	**0.672**	454.3	**0.815**	14402	**0.815**	14400
J120	390	0.839	66.2	**0.820**	113.9	0.809	1516.2	**0.714**	2111.4	**0.832**	14402	**0.832**	14423
total	782	0.734	26.6	**0,723**	232.1	0.711	447.5	**0.622**	682.6	**0.670**	25825	0.671	25083

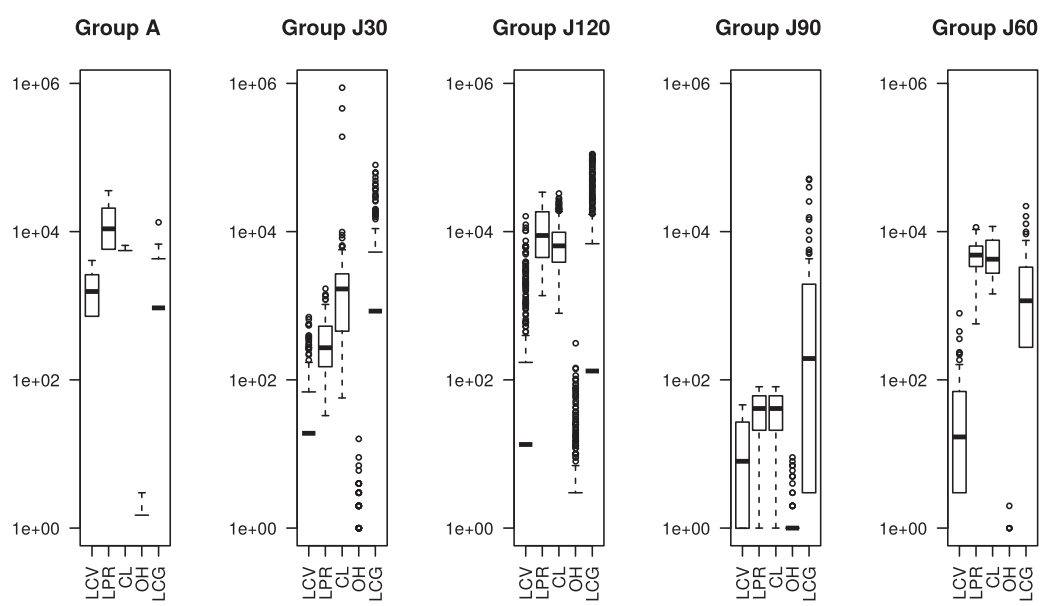


Fig. 3. Boxplots of the number of cuts for different types and benchmark datasets to each separation strategy.

Finally, the quantities of both cliques and lifted precedence cuts are relatively high. However, their separation requires more time.

5.2.2. Results removing cut families

While the previous experiments explored the impact of adding each cut type, we now explore the impact of omitting each cut type. To this end, we use all cut types, separated in parallel in the cutting plane, and then individually remove each type for the benchmark datasets SMRCPSP and MMRCPSP. Table 6 summarizes the results of these experiments, where column pair All Cuts represents the strategy using all cut families, and the other column pairs represent the strategy without each of the cut types.

Table 6
Average integrality gaps and the average computing times (sec.) obtained by the cutting plane using all cut types and then removing one cut type at a time with time limit of 4 h.

		All Cuts		-{LCV}		-{LPR}		-{CL}		-{OH}		-{SCG}	
group	n	gap	time	gap	time	gap	time	gap	time	gap	time	gap	time
J30	245	**0.458**	13546	0.471	13631	0.485	13640	0.459	13645	0.461	13661	0.518	8
J60	57	**0.702**	14402	0.703	14401	0.798	14401	**0.702**	14401	**0.702**	14402	0.71	107
J90	80	0.666	14403	0.667	14403	0.8	14402	**0.665**	14403	0.666	14403	0.669	545
J120	390	0.668	14409	0.713	14436	0.774	14425	**0.667**	14429	0.67	14429	**0.667**	4779
Total	772	0.624	14190	0.639	14218	0.714	14217	**0.623**	14219	0.625	14224	0.641	1360

Table 7
Average number of iterations and the average computing times (sec.) obtained by the cutting plane comparing with removing some others cuts with time limit of 4 h.

		All Cuts		-{LCV}		-{LPR}		-{CL}		-{OH}		-{SCG}	
group	n	round	time	round	time	round	time	round	time	round	time	round	time
J30	245	2077	13546	1946	13631	1796	13640	2486	13645	2035	13661	12	8
J60	57	618	14402	618	14401	877	14401	832	14401	452	14402	16	107
J90	80	294	14403	300	14403	476	14402	401	14403	293	14403	22	545
J120	390	265	14409	271	14436	347	14425	469	14429	284	14429	170	4779
Total	772	814	14190	784	14218	874	14217	1047	14219	766	14224	55	1360

Table 8
Average integrality gaps and the average computing times (sec.) obtained by the cutting plane comparing with removing some others cuts with time limit of 24 h.

	All Cuts		-{CL&OH}		-{SCG}	
instance	gap	time	gap	time	gap	time
A-1	**0.000**	52.5	0.875	0.7	0.875	0.4
A-2	**0.781**	86015.2	0.924	4.2	0.924	5.6
A-3	**0.000**	86368.5	**0.000**	11.0	**0.000**	9.4
A-4	**0.316**	86001.1	0.327	31.1	0.327	104.2
A-5	**0.263**	86010.4	0.267	832.2	0.266	1280.4
A-6	**0.313**	86089.2	0.314	2130.2	0.314	2660.6
Total	**0.279**	71756.2	0.451	501,6	0.451	676.8

By removing the LPR cuts, results worsen by about 13%. The results also get a little worse by removing OH, LCV, and SCG respectively. The results using all cuts performed generally well. In addition, removing the clique cuts further improved the gaps on larger instances. This can be explained by the fact that separating cliques requires more time. When these cuts are removed the numbers of iterations increases, as one can observe in Table 7. Based on these results, one may define a new hierarchical sequence of the cut types based on the gap increase when the cut type is removed: LPR, SCG, LCV, OH, and CL.

Table 8 shows the results for the same experiments for the MMRCMPSP. Again, column All Cuts represent the strategy were all cut types are used. Instead of reporting the results when each of the other cut types is removed, we only report on removing the cut types that consumed most of the computing time: cliques and odd-holes cuts, or the strengthened CG cuts. The bold numbers show the best average integrality gaps achieved for instances that did not run out of memory. Even though the results are better on average when using all cuts, by removing the cuts, the time was reduced significantly. Even though using all cut types significantly increases the computing times, the gap improvement on some of the instances may still justify their use.

To analyze the number of generated cuts for all separation procedures, we also computed the number of non-repeated cuts to the previous experiments (All Cuts). Fig. 4 shows the boxplots representing the number of cuts for different types and benchmarks.

Fig. 4 shows that LPR cuts are effective for all datasets mainly in the case of the SMRCPSP, where it found, on average, more cuts than others. For the MMRCMPSP, the LPR cuts are still the most effective ones. CL and SCG cuts seem to be effective mainly for instances with multiple modes. The average number of generated LCV cuts seems to be stable for all benchmarck dataset. Few OH cuts are found for all benchmarck datasets.

After analyzing the best strategy for the MMRCPSP obtained through experiments on the PSBLIB instances, we run the experiment with all cuts for the instances from MMLIB, comparing the LP relaxation and the strengthened linear relaxation. The results are presented in Table 9.

The results show that the cutting plane was very effective for all benchmark datasets, improving the average values by approximately 15%. As in earlier comparisons, the SLR slightly improves the integrality gaps for MMLIB instances too. However, 135 of the JA11 instances ran out of memory, which have been removed from the results presented in the table to provide a fair comparison.

5.3. Branch & cut experiments

We now explore how the different cut types can be added dynamically in a Branch-and-Cut manner, to improve the solution process using the general purpose MILP solver Gurobi. Experiments were executed in order to compare the results achieved by solving the model with the inclusion of cuts into the root plus precedence cuts into a callback[7] procedure when the lower bound is improved and the results achieved by solving the model without cuts just with the preprocessing input data and Gurobi cuts.

Table 10 summarizes the integrality gaps[8] (average and standard deviation), for open instances with $\alpha > 0$ from PSPLIB and MISTA. The experiments for our approach have been limited to 24 h of computing time. For Gurobi cuts, the first experiments have been limited to 24 h and the second have been limited to 48 h, in order to ensure a fair comparison since we not consider the time spent to insert all cuts into the root node for our approach. Initial solutions available at MISTA website were inserted only for the A dataset.

[7] a callback is a user function that is called periodically by the Gurobi optimizer in order to allow the user to query or modify the state of the optimization Gurobi Optimization (2016).

[8] the integrality gaps is computed as in the previous experiments, since it was not possible to generate an incumbent solution for all instances, so it is not possible to obtain the optimality gap for some instances.

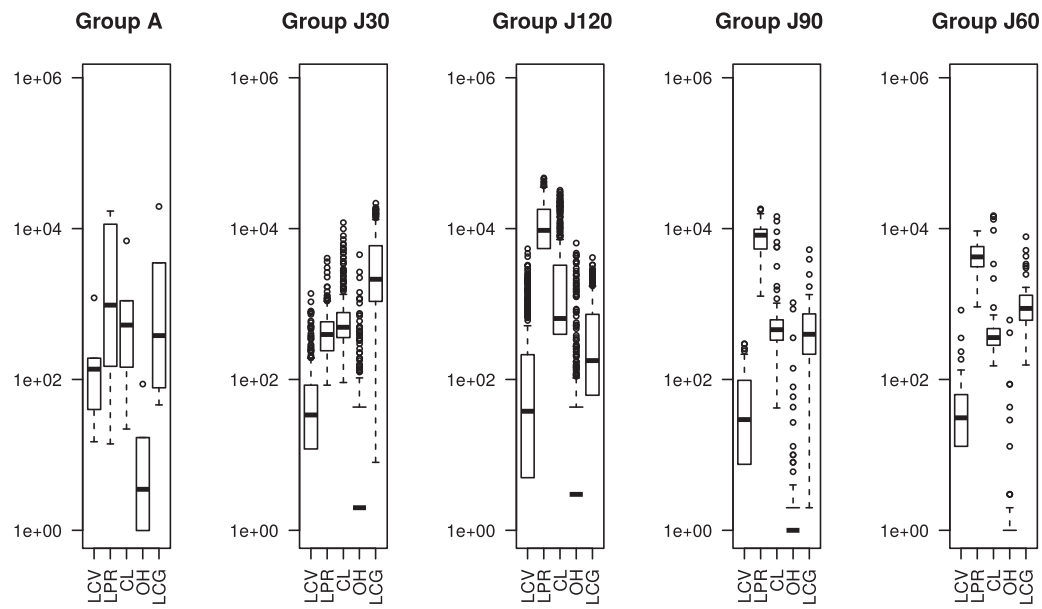


Fig. 4. Boxplots of the number of cuts for different types and benchmark datasets to all separation strategy together.

Table 9
Average integrality gaps and the average computing times (sec.) obtained by the cutting plane comparing with the LP relaxation and its strengthening with time limit of 4 h.

		LR			SLR			+{All Cuts}		
group	*n*	gap	std	time	gap	std	time	gap	std	time
J50	540	0,750	0,309	0,6	0,749	0,309	6,4	**0,654**	0,367	9669,4
J100	540	0,768	0,293	11,0	0,768	0,293	35,6	**0,688**	0,354	11536,6
JAll50+	1616	0,586	0,250	6,6	0,584	0,277	30,1	**0,463**	0,227	13746,8
Jall100+	1489	0,644	0,272	54,8	0,644	0,272	144,0	**0,530**	0,265	14221,9
Total	4185	0,687	0,281	18,2	0,686	0,288	54,0	**0,584**	0,303	12293,7

Table 10
Average and standard deviation for the integrality gaps obtained by the B&C with all cuts at root plus LPR in the callback procedure.

		Gurobi cuts				+{Our approach}	
		gap (24h)		gap (48h)		gap (24h)	
group	*n*	avg	std	avg	std	avg	std
A	10	0.210	0.165	0.158	0.139	**0.139**	**0.124**
J30	245	0.007	0.022	0.006	0.022	**0.005**	**0.020**
J60	57	0.252	0.115	0.237	0.103	**0.184**	**0.084**
J90	80	0.268	0.162	0.253	0.139	**0.201**	**0.084**
J120	390	0.299	0.253	0.292	0.252	**0.249**	**0.224**

Table 11
Solutions obtained by the B&C with all cuts at root plus LPR in the callback procedure.

		Gurobi cuts			+{Our approach}		
group	*n*	opt	fea	inf	opt	fea	inf
A	10	**4**	6	0	**4**	6	0
J30	641	530	6	**89**	**553**	**15**	**89**
J60	79	24	0	0	**26**	**2**	0
J90	105	**27**	0	0	**27**	0	0
J120	514	160	2	0	**181**	**3**	0

By analyzing Table 10 we can see that better results are obtained with the introduction of cuts into the root and with the LPR cut into the callback procedure. The average integrality gaps are reduced. Also, for some particular instances like *j*12049_8, *j*12021_1, *j*12022_8, *j*3037_1, *j*3037_7, *j*6046_5, *j*6030_2, among others, the average optimality gap achieved within 24 h of computing time was quite low (0.09%) when adding the cuts. Almost all open instances with $\alpha = 0$ from PSPLIB have been easily solved to optimality, except for some instances in set with 120 jobs.

We now explore how the two settings compare throughout the optimization process to find feasible solutions and to prove optimality. Table 11 summarizes the results for datasets from MISTA and PSPLIB for the three problem variants, also including those instances with $\alpha = 0$. Comparing to Toffolo et al. (2016), we solve to optimality, for the first time, 1 instance from A set. Comparing to Schnell and Hartl (2017) we solve the J30 set, for the first time, 13 instances to optimality, and prove infeasibility for 89 of the instances.

The results indicate that our approach has been able to prove optimality for more instances than using Gurobi without our cuts. For some specific instances the optimal value is found only when cuts are added to the model. In addition, Gurobi solver proves that some instances are infeasible.

Table 12 summarizes the results for our approach on the MMLIB datasets. The table shows the number of instances that have been solved to optimality (opt) and the number of instances for which a feasible solution has been found, but optimality has not been proven (fea). Further, the table shows the number of instances for which our solutions improve upon those reported by the website[9] (imp) and those reported in Schnell and Hartl (2017), as well the number of instances for which optimality has been proven for the first time (new opt).

[9] http://mmlib.eu/solutions.php.

Table 12
Solutions obtained by the B&C with all cuts at root plus LPR in the callback procedure.

		all cuts at root + LPR callback				
group	n	opt	new opt	fea	imp	imp Schnell and Hartl (2017)
J50	540	450	29	32	0	48
J100	540	410	48	43	7	114
JAll50+	1620	689	252	159	59	401
JAll100+	1620	482	177	205	174	440

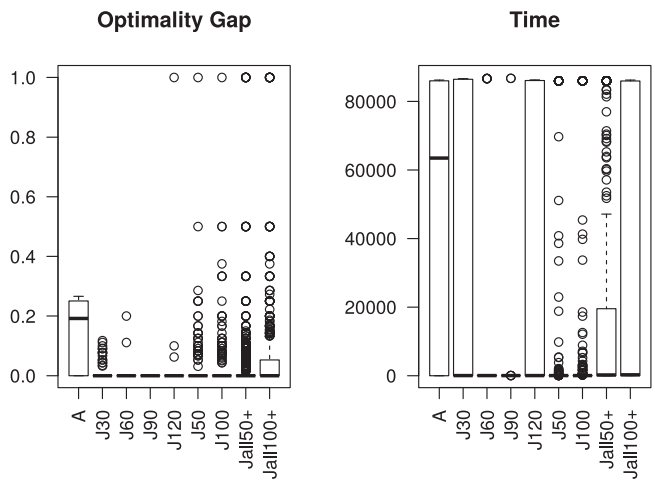


Fig. 5. Boxplots with informations about optimality gap and computing time (sec.) of instances from PSPLIB, MISTA and MMLIB .

Fig. 5 presents boxplots of the optimality gaps and computing times for our B&C approach for instances from PSPLIB, MISTA and MMLIB for which feasible or optimal solutions have been found.

The results presented in the first figure suggest that the gap values equals to 0 are represented by the median in the box plot for almost all datasets, except for the A and Jall100+ datasets. The high outliers for the datasets J120, J50, J100, Jall50+ and Jall100+ indicate that the presented algorithms still require improvement in order to deliver robust results on all instances. Analyzing the computing times it can be observed that our approach quickly proves optimality for all but dataset A. Finally, we also note that some of the instances between the second and third quartile hit the time limit.

In summary, optimality was proven for the first time, for 247 instances from PSPLIB, for 1 instance from MISTA and for 506 instances from MMLIB, totalizing 754 instances. The LP and solution files for all benchmark datasets are available for download at http://professor.ufop.br/janniele/downloads.

6. Conclusion and future works

In this paper, new mixed-integer linear programming based methods were proposed to improve the linear programming relaxation of compact formulations for the Resource Constrained Project Scheduling Problem. All methods were extensively evaluated in three RCPSP variants.

An effective preprocessing procedure to strengthen renewable resources constraints was devised. This procedure was capable of improving the lower bounds produced at the root node without any increase in the size of the linear programs.

A parallel cutting plane algorithm was developed including five families of cuts: lifted precedence and cover cuts, cliques, odd-holes and strengthened Chvátal-Gomory cuts. A dense conflict graph, considering feasibility and optimality conditions, was created at each iteration and used by these cut generators in strengthening procedures. All cuts contributed to improve the lower bounds, specially when they are together in the cutting plane. However, the lifted precedence cuts were the most effective for all variants. The strengthened Chvátal-Gomory cuts were specially effective in a group of multi-project instances. These results indicate that an instance feature based tuning of the cut generators may be beneficial.

With the improved linear programming formulations produced with our methods, 754 open instances from literature were solved for the first time: 247 instances from PSPLIB, 1 instance from the MISTA Challenge and for 506 instances from MMLIB.

The method still has room for improvement. These techniques could be hybridized with constraint propagation, as proposed in previous studies. The separation of some inequalities, such as the strengthened Chvátal-Gomory cuts and the re-optimization of the large linear programs are still quite expensive. The continuous improvements in future MILP solvers will likely speed up these two steps. Finally, the development of a complete B&C with improved branch and node selection strategies for the RCPSP, including our preprocessing and cutting planes routines, is also a promising future path.

Acknowledgements

The authors thank UFOP, CNPq (student with scholarship - Brazil), FAPEMIG, and Compute Canada for supporting this research.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.cor.2019.104782.

References

Applegate, D., Cook, W., 1991. A computational study of the job-shop scheduling problem. ORSA J. Comput. 3, 149–1556.
Artigues, C., 2017. On the strength of time-indexed formulations for the resource–constrained project scheduling problem. Oper. Res. Lett. 45, 154–159.
Artigues, C., Demassey, S., Néon, E., 2008. Resource-Constrained Project Scheduling: Models, Algorithms, Extensions and Applications. ISTE Ltd and John Wiley & Sons, Inc.
Atamtürk, A., Nemhauser, G.L., Savelsbergh, M.W.P., 2000. Conflict graphs in solving integer programming problems. Eur. J. Oper. Res. 121 (1), 40–55.
Balas, E., 1975. Facets of the knapsack polytope. Math. Program. 8 (1), 146–164.
Balas, E., Zemel, E., 1978. Facets of the knapsack polytope from minimal covers. SIAM J. Appl. Math. 34 (1), 119–148.
Baptiste, P., Demassey, S., 2004. Tight lp bounds for resource constrained project scheduling. OR Spectr. 26 (2), 251–262.
Blazewicz, J., Lenstra, J., Rinnooy Kan, A., 1983. Scheduling subject to resource constraints: classification and complexity. Discrete Appl. Math 5, 11–24.
Boyd, E., 1992. Fenchel cutting planes for integer programming. Oper. Res. 42, 53–64.
Boyd, E., 1994. Solving 0/1 integer programs with enumeration cutting planes. Ann. Oper. Res. 50, 61–72.
Brito, S.S., Santos, H.G., Poggi, M., 2015. A computational study of conflict graphs and aggressive cut separation in integer programming. Electron. Notes Discrete Math. 50, 355–360.
Brucker, P., Knust, S., Schoo, A., Thiele, O., 1998. A branch and bound algorithm for the resource-constrained project scheduling problem. Eur. J. Oper. Res. 107, 272–288.
Cavalcante, C., de Souza, C., Savelsbergh, M., Y, W., Wolsey, L.A., 2001. Scheduling projects with labor constraints. Discrete Appl. Math. 112 (1), 27–52.

Chakrabortty, R.K., Sarker, R.A., Essam, D.L., 2015. Resource constrained project scheduling: a branch and cut approach. In: International Conference on Computers and Industrial Engineering, Vol. 45, pp. 552–559.
Christofides, N., Alvarez-Valdes, R., Tamarit, J., 1987. Project scheduling with resource constraints: a branch and bound approach. Eur. J. Oper. Res. 29, 262–273.
Chvátal, V., 1973. Edmonds polytopes and a hierarchy of combinatorial problems. Discrete Math. 4, 305–337.
de Souza, C.C., Wolsey, L.A., 1997. Scheduling Projects with Labour Constraints. Technical report. Instituto de ComputaÃ§Ã£o, Universidade Estadual de Campinas.
Demassey, S., Artigues, C., Michelon, P., 2005. Constraint propagation based cutting planes : an application to the resource-constrained project scheduling problem. INFORMS J. Comput. 17, 52–65.
Demeulemeester, E., Herroelen, W., 1992. Recent advances in branch-and-bound procedures for resource-constrained project scheduling problems. In: Paper Presented at the Summer School on Scheduling Theory and its Applications, pp. 1–32.
Demeulemeester, E.L., Herroelen, W.S., 2002. Project Scheduling: A Research Handbook. Kluwer Academic Publishers.
Fischetti, M., Lodi, A., 2007. Optimizing over the first Chvátal closure. Math. Program. 110 (1), 3–20.
Garey, M.R., Johnson, D.S., 1979. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman & Co., New York, NY, USA.
Gu, Z., Nemhauser, G., Savelsbergh, M., 2000. Sequence independent lifting in mixed integer programming. J. Comb. Optim. 4 (1), 109–129.
Gurobi Optimization, I., 2016. Gurobi Optimizer: Reference Manual.
Hardin, J.R., Nemhauser, G.L., Savelsbergh, M.W., 2008. Strong valid inequalities for the resource-constrained scheduling problem with uniform resource requirements. Discrete Optim. 5, 19–35.
Johnson, E., Kostreva, M., Suhl, U., 1985. Solving 0–1 integer programming problems arising from large scale planning models. Oper. Res. 33, 803–819.
Kelley Jr, J., Walker, M.R., 1959. Critical-path planning and scheduling. In: Eastern Joint IRE-AIEE-ACM Computer Conference. ACM, pp. 160–173.
Kolisch, R., 1995. Project Scheduling under Resource Constraints: Efficient Heuristics for Several Problem Classes. Number IX, 212. Production and Logistics, first ed. Physica-Verlag Heidelberg.
Kolisch, R., Sprecher, A., 1996. PSPLIB - a project scheduling problem library. Eur. J. Oper. Res. 96, 205–216.
Kone, O., Artigues, C., Lopez, P., Mongeau, M., 2011. Event-based MILP models for resource-constrained project scheduling problems. Comput. Oper. Res. 38, 3–13.
Land, A.H., Doig, A.G., 2010. An Automatic Method for Solving Discrete Programming Problems. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 105–132.
Letchford, A.N., Marzi, F., Rossi, F., Smriglio, S., 2016. Strengthening Chvátal-Gomory cuts for the stable set problem. In: Combinatorial Optimization. Springer International Publishing, pp. 201–212.
Liu, M., Shan, M., Wu, J., 2014. Multiple R&D projects scheduling optimization with improved particle swarm algorithm. Sci. World J. 2014. Article ID 652135, 7 pages, doi: 10.1155/2014/652135.
Liu, S.-S., Wang, C.-J., 2008. Resource-constrained construction project scheduling model for profit maximization considering cash flow. Autom. Constr. 17 (8), 966–974.
Mingozzi, A., Maniezzo, V., Ricciardelli, S., Bianco, L., 1998. An exact algorithm for the resource-constrained project scheduling problem based on a new mathematical formulation. Manage. Sci. 44 (5), 714–729.
Nemhauser, G.L., Vance, P.H., 1994. Lifted cover facets of the 0–1 knapsack polytope with GUB constraints. Oper. Res. Lett. 16 (5), 255–263.
Padberg, M.W., 1973. On the facial structure of set packing polyhedra. Math. Program. 5 (1), 199–215.
Patterson, J., Huber, W., 1974. A horizon-varying zero-one approach to project scheduling. Manage. Sci. 20, 990–998.
Pritsker, A., Watters, L., Wolfe, P., 1969. Multi project scheduling with limited resources: a zero-one programming approach. Manage. Sci. 3416, 93–108.
Riedler, M., Jatschka, T., Maschler, J., Raidl, G.R., 2020. An iterative time-bucket refinement algorithm for a high resolution resource-constrained project scheduling problem. Int. Trans. Oper. Res. 27, 573–613. doi:10.1111/itor.12445.
Sankaran, J.K., Bricker, D.L., Juang, S., 1999. A strong fractional cutting plane algorithm for resource-constrained project scheduling. Int. J. Ind. Eng. 6, 99–111.
Santos, H.G., Toffolo, T.A.M., Gomes, R.A.M., Ribas, S., 2016. Integer programming techniques for the nurse rostering problem. Ann. Oper. Res. 239 (1), 225–251.
Schnell, A., Hartl, R.F., 2017. On the generalization of constraint programming and boolean satisfiability solving techniques to schedule a resource-constrained project consisting of multi-mode jobs. Oper. Res. Perspect.s 4, 1–11.
Schwindt, C., Zimmermann, J., 2015. Handbook on project management and scheduling. International Handbooks on Information Systems, Vol. 1. Springer International Publishing.
Toffolo, T., Santos, H., Carvalho, M., Soares, J., 2016. An integer programming approach to the multimode resource-constrained multiproject scheduling problem. J. Scheduling 19, 295–307.
Van Peteghem, V., Vanhoucke, M., 2014. An experimental investigation of metaheuristics for the multi-mode resource-constrained project scheduling problem on new dataset instances. Eur. J. Oper. Res. 235 (1), 62–72.
Wauters, T., Kinable, J., Smet, P., Vancroonenburg, W., Vanden Berghe, G., Verstichel, J., 2016. The multi-mode resource-constrained multi-project scheduling problem. J. Scheduling 19, 271–283.
Wolsey, L., 1998. Integer Programming. Wiley - Interscience Series in Discrete Mathematics and Optimization.
Zhu, G., Bard, J., Yu, G., 2006. A branch-and-cut procedure for the multimode resource-constrained project-scheduling problem. INFORMS J. Comput. 18 (3), 377–390.